

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Новгородский государственный университет имени Ярослава Мудрого»
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ

УТВЕРЖДАЮ

Зам. директора по УМ и ВР

_____ Л.Н. Иванова

«___» _____ 2025 г.

СОЗДАНИЕ МНОГОПОЛЬЗОВАТЕЛЬСКОЙ ИГРЫ «SUPER BOMBERMAN»
С ВНЕДРЕНИЕМ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА ДЛЯ
ПРОТИВНИКОВ

Пояснительная записка к дипломному проекту по специальности
09.02.07 Информационные системы и программирование
ПТК. ДП 1991 02. 000ПЗ

Согласовано:

Консультант по спец. части

_____ Л.Н. Цымбалюк

«___» _____ 2025 года

Консультант по экон. части

_____ Г.В. Лебедева

«___» _____ 2025 года

Нормоконтроль

_____ А.М. Чернега

«___» _____ 2025 года

Руководитель

_____ В.В. Бурбах

«___» _____ 2025 года

Выполнил:

обучающийся группы 1991

_____ И.Б. Богданов

«___» _____ 2025 года

Заместитель директора по УПР

_____ А.М. Чернега

«___» _____ 2025 года

Содержание

Содержание	2
1 Общая часть	5
1.1 Постановка задачи.....	5
1.2 Обоснование проектных решений.....	9
1.3 Обзор и анализ существующих программных систем	12
2 Специальная часть.....	13
2.1 Анализ задачи	13
2.2 Описание логической структуры.....	19
2.3 Описание работы программы	26
2.4 Руководство оператора	28
3 Экономическая часть	33
3.1 Расчёт заработной платы с отчислениями на социальное страхование	33
3.2 Расчёт стоимости материалов и лицензионного обеспечения	37
3.3 Расчёт накладных расходов	38
3.4 Составление и расчёт цены реализации программного продукта	40
Заключение	41
Список литературы	42
Приложение А	43
Приложение Б	48
Приложение В.....	52
Приложение Г	55

Видеоигра — это специальная компьютерная программа, предназначенное для проведения игровых сессий пользователем. Она может включать в себя разнообразные жанры, от стратегий и головоломок до экшенов и симуляторов. Видеоигры предоставляют игрокам уникальную возможность погрузиться в виртуальный мир, где они могут взаимодействовать с окружающей средой, персонажами и другими игроками.

Многопользовательская игра — это вид видеоигры, в которой несколько игроков могут участвовать одновременно, взаимодействуя друг с другом внутри игрового мира. Такие игры могут предоставлять различные формы соревнований или сотрудничества между игроками, создавая уникальный игровой опыт.

«Super Bomberman» — это культовая видеоигра, выпущенная в 1985 году для игровой консоли «Nintendo Entertainment System». В этой игре игроки управляют персонажами, размещая бомбы, чтобы уничтожить противников и преодолевать препятствия на уровнях. Она стала популярной благодаря захватывающему мультиплееру и интересному геймплею.

Искусственный интеллект активно применяется в игровой индустрии для создания интеллектуального поведения неигровых персонажей и противников. Использование искусственного интеллекта у противников позволяет сделать игровой процесс более увлекательным, повышая интерес игроков.

Актуальность исследования имеет несколько причин:

- растущий интерес к многопользовательским играм и возможность игры в реальном времени с друзьями и другими игроками;
- пародия на популярную игру «Super Bomberman» предоставляет уникальную возможность привлечь аудиторию благодаря знакомым механикам и стилизации под известную франшизу;
- многопользовательские игры, представляют собой отличную возможность для создания сообщества игроков вокруг определенного проекта. это способствует формированию активной аудитории.

Поэтому разработка многопользовательских игр с искусственным интеллектом для противников является актуальной задачей, позволяющей создать интересный продукт, доступный широкой аудитории.

Целью данной выпускной квалификационной работы является создание многопользовательской онлайн-игры «Super Bomberman» с внедрением искусственного интеллекта для управления поведением игровых противников. Планируется обеспечить игрокам увлекательный игровой опыт, возможность играть в реальном времени и запуск игры в любом браузере, что расширит потенциальную аудиторию.

Объект исследования — многопользовательская онлайн-игра «Super Bomberman».

Предмет исследования — процесс создания интересного игрового опыта на основе классической франшизы, адаптированной к онлайн-формату, с использованием искусственного интеллекта.

Цель исследования: создание программного продукта, который позволит пользователям взаимодействовать в реальном времени, наслаждаться игровым опытом и проводить время в захватывающей игровой среде.

Задачами выпускной квалификационной работы в связи с указанной целью являются:

- знакомство с предметной областью;
- написание концепт-документа игры;
- подготовка графики и интерфейса игры;
- проектирование и разработка алгоритмов;
- написание программного кода;
- тестирование и публикация игры.

1 Общая часть

1.1 Постановка задачи

1.1.1 Обоснование необходимости разработки

На сегодняшний день оригинальная игра «Super Bomberman», выпущенная в 1985 году, не имеет аналогов с многопользовательской поддержкой. Несмотря на то, что разработчики серии уже выпустили продолжения в трехмерном формате, сообщество игроков по-прежнему любит классическую пиксельную версию. Это указывает на устойчивый интерес к классическому стилю игры, её оригинальной механике и визуальному оформлению.

Многопользовательский режим является ключевым фактором в привлечении игроков и создании активного сообщества вокруг игры. Возможность играть с друзьями и другими игроками в режиме реального времени увеличивает интерес к игре. Учитывая популярность классической версии «Super Bomberman» и высокий потенциал многопользовательской игры, разработка подобного проекта оправдана и имеет значительные перспективы.

Также важно обеспечить поддержку офлайн-режима с ботами, управляемыми алгоритмами искусственного интеллекта. Благодаря этому игроки смогут тренироваться и наслаждаться классическим геймплеем без других игроков, а интеллектуальные противники создадут достаточно реалистичные условия для одиночных матчей.

1.1.2 Техничко-математическое описание задачи

Для создания используется игровой движок Unity с WebGL-билдом, что позволяет запускать клиентскую часть в браузере. Логика реализована на языке C#, а результаты компиляции Unity генерируют код WebAssembly и JavaScript, который обёрнут в HTML и воспроизводится в любом современном веб-браузере. Сетевая часть построена на Photon PUN поверх протокола UDP: протокол HTTP применяется только при создании и поиске игровых комнат, тогда как синхронизация позиций, состояния бомб и бонусов идёт по UDP-сообщениям для минимизации задержек в реальном времени.

Основную математическую модель составляет дискретная двумерная сетка тайлмапа: каждая клетка задаётся двумя целыми координатами x и y , и именно по этим координатам определяются положения игроков, бомб и бонусов. Область поражения взрыва вычисляется обходом по четырём направлениям от исходной клетки, при этом распространение взрыва останавливается на непреодолимых плитках или после достижения заданного радиуса. Для расчёта маршрутов у ботов используется алгоритм поиска пути A^* с эвристической функцией Манхэттена, что позволяет выбирать кратчайший путь к цели и одновременно избегать зон, в которых скоро произойдёт взрыв.

1.1.3 Характеристика бизнес-процессов

Бизнес-процесс «Организация игрового процесса» описывает все этапы от выбора режима игры до завершения матча и формирования финальной статистики. Сначала игрок находится на экране настроек: здесь он выбирает количество участников, число раундов и карту. Эти действия пользователя формируют режим и параметры будущей игровой сессии, то есть задают, сколько людей будет играть, на какой карте и сколько всего раундов должно пройти.

После того как параметры заданы, начинается этап «Запуск игрового сеанса». На вход этого этапа поступают выбранные игроком настройки и сама карта, а также свод правил игры. Блок подключения создает или присоединяется к сетевой комнате, распределяет игроков по стартовым позициям и загружает игровую сцену. В процессе работы этого этапа пользователь выполняет различные игровые действия: перемещается, устанавливает бомбы и собирает бонусы. Одновременно в этот же блок поступают синхронизационные события от других участников — информация о том, кто куда переместился, где была поставлена и взорвана бомба, какие бонусы подобраны. Результатом этапа «Запуск игрового сеанса» становятся «Данные игрового сеанса» — текущее состояние всех игроков, бомб и бонусов в раунде.

Когда все запланированные раунды сыграны, управление переходит к блоку «Завершение игрового сеанса». На вход этого этапа поступают «Данные игрового сеанса». Здесь происходит подсчет победителей каждого раунда, обновление общего счета и формирование итоговой «Статистики матча», в которой фиксируется, сколько раундов выиграл каждый игрок и сколько раундов было сыграно в целом. Параллельно накапливаются «Скрытые логи» — это подробный хронологический отчет о событиях, который затем используется для отладки и анализа работы игры. В зависимости от результатов матч либо завершается и возвращает всех игроков обратно в лобби с показом финальных результатов, либо автоматически перезагружается для следующих раундов, если их еще не все сыграны.

Диаграмма IDEF0 с обзором всего процесса приведена на рисунке 1.

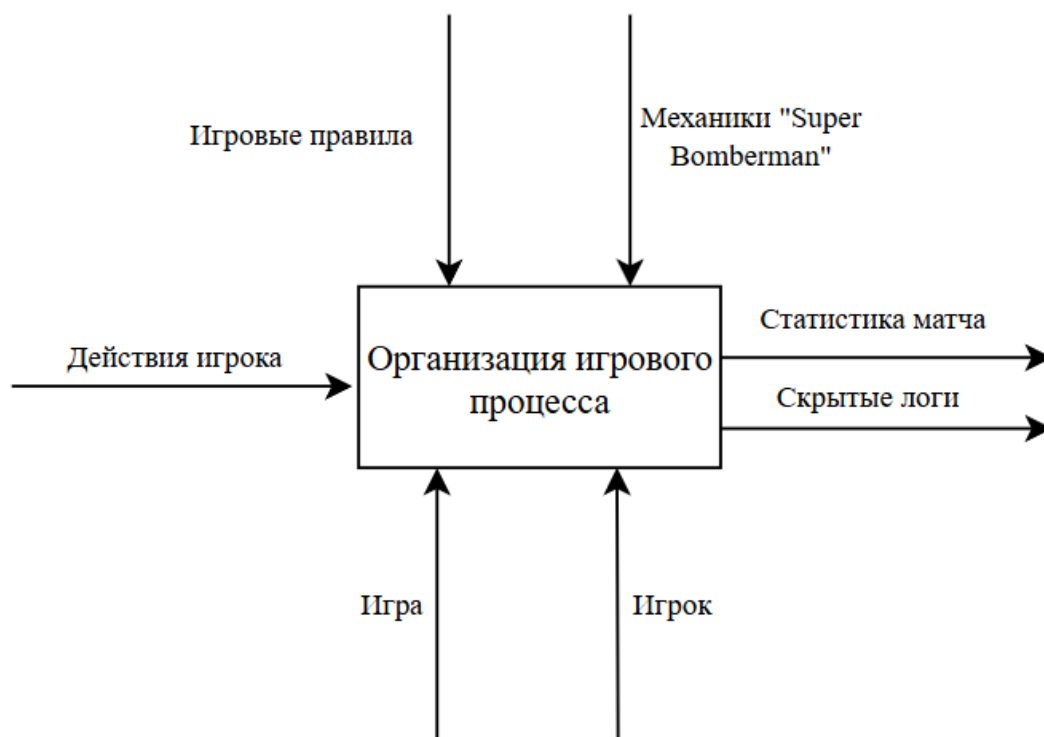


Рисунок 1 – IDEF0 организация игрового процесса

Декомпозированная модель диаграммы IDEF0 организации игрового процесса с тремя подпроцессами «Выбор и настройка режима игры», «Запуск игрового сеанса» и «Завершение игрового сеанса» показана на рисунке 2.

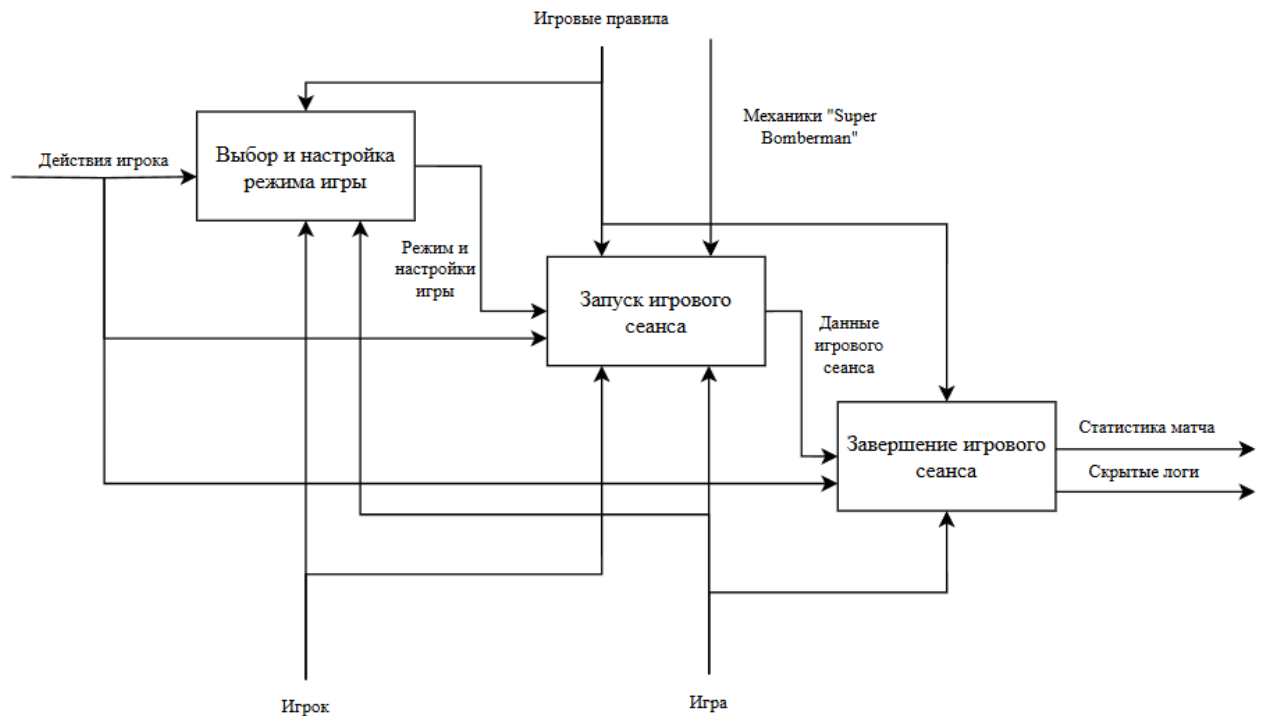


Рисунок 2 – Декомпозиция IDEF0 организация игрового процесса

1.1.4 Требования к программе

Для запуска веб-версии игры BombersOnline требуется:

- веб-браузер с поддержкой WebGL последней версии, установленный на устройстве;
- низкая пинг-латентность, не более 100 мс до сервера Photon PUN для обеспечения синхронизации состояний в реальном времени;
- стабильное интернет-соединение с пропускной способностью не менее 1 Мбит/с для обеспечения бесперебойного обмена данными в режиме реального времени.

Наличие версии в формате WebGL позволяет запускать игру практически на любом современном компьютере, что значительно расширяет охват аудитории и делает игру более доступной.

1.2 Обоснование проектных решений

1.2.1 Обоснование выбора языков программирования

Unity поддерживает несколько языков программирования, включая C#, JavaScript и Boo. Однако, наиболее популярным языком программирования для Unity является C#.

C# — это объектно-ориентированный язык, который хорошо согласуется с принципами проектирования модульного и поддерживаемого программного обеспечения. Он поддерживает такие концепции, как инкапсуляция, наследование и полиморфизм, обеспечивая чистые и организованные структуры кода.

Причины почему C# является лучшим выбором для работы с Unity:

- широкое распространение: C# является одним из самых популярных языков программирования в мире, и многие разработчики уже знакомы с ним. Это упрощает процесс обучения и сокращает время настройки проекта;
- интеграция: Unity имеет прямую интеграцию с C#, что облегчает процесс разработки и отладки;
- производительность: C# является высокопроизводительным языком программирования, что делает его идеальным для создания игр с высокими требованиями к производительности;
- большое сообщество: C# имеет большое сообщество разработчиков, способных предоставить поддержку и советы, что может быть полезно при разработке игр на Unity;
- легкость в использовании: C# является относительно простым языком программирования, и новички могут быстро освоить его основы.

C# является лучшим выбором для работы с Unity, но выбор языка программирования также зависит от индивидуальных потребностей и опыта разработчика.

1.2.2 Инструментальные средства

Для разработки проекта будет использоваться игровой движок Unity.

Unity — это мощный и популярный движок для разработки игр. Unity позволяет создавать приложения, работающие под более чем 20 различными операционными системами, включающими персональные компьютеры, игровые консоли, мобильные устройства, интернет-приложения и другие.

Unity имеет ряд преимуществ в разработке браузерных игр:

- кроссплатформенность: Unity позволяет создавать игры для множества платформ, включая браузеры, что обеспечивает максимальную охват аудитории игроков;
- высокая производительность: Unity оптимизирован для работы с графикой и имеет множество инструментов для улучшения производительности игры, что позволяет создавать качественные игры с минимальными нагрузками на систему;
- большое сообщество и экосистема: Unity имеет огромное сообщество разработчиков и множество готовых решений, библиотек и плагинов для ускорения разработки, и повышения качества игры;
- легкость в использовании: Unity имеет интуитивно понятный интерфейс и множество инструментов для упрощения разработки, что позволяет сократить время разработки и повысить качество игры;
- множество возможностей: Unity поддерживает множество форматов моделей, текстур, аудио и других элементов игры, что позволяет создавать качественные и разнообразные игры;
- большое количество готовых ресурсов: Unity имеет множество готовых ресурсов, которые можно использовать для создания игры, таких как готовые компоненты, модели, текстуры, шейдеры и другие.

Помимо Unity используются инструментальные средства, такие как:

- Figma – это онлайн-инструмент для дизайна интерфейсов и создания прототипов;

- Balsamiq Mockups – это инструмент для быстрого создания прототипов пользовательского интерфейса, предназначенный для дизайнеров, разработчиков и менеджеров проектов;

- Aseprite – это инструмент для создания пиксельной анимации и графики. Он был разработан для аниматоров и художников, которые работают с 2D-графикой, такой как игры, мультфильмы, комиксы и другие виды искусства;

- Git – это распределенная система контроля версий, которая используется разработчиками для управления изменениями в исходном коде проекта.

1.2.3 Обоснование выбора среды программирования

Для разработки проекта была выбрана интегрированная среда разработки Visual Studio.

Visual Studio — это мощная среда разработки, созданная компанией Microsoft, поддерживающая различные платформы. Она предоставляет широкий набор инструментов, таких как подсветка синтаксиса, автодополнение кода, отладка, тестирование и интеграция с системами управления версиями.

Среда поддерживает множество языков программирования, включая C#, C++, Python и JavaScript, что делает её универсальным инструментом для разработки. Благодаря множеству расширений и плагинов, Visual Studio позволяет адаптировать функциональность среды под конкретные задачи, улучшая процесс разработки и повышая её эффективность.

Особое преимущество Visual Studio заключается в её тесной интеграции с игровым движком Unity, что официально рекомендуется для разработки игр на этой платформе. Visual Studio облегчает написание и отладку кода на языке C#, который является основным языком программирования в Unity, обеспечивая бесшовную работу с игровым движком. Кроме того, среда поддерживает интеграцию с сервисами Microsoft, такими как Azure и GitHub, что упрощает развертывание и управление проектами в облачной среде.

Visual Studio также пользуется широкой популярностью среди разработчиков благодаря активному сообществу и обширной поддержке. Это

обеспечивает доступ к множеству ресурсов, обучающих материалов и решений для различных проблем, что значительно упрощает процесс разработки.

Хотя существуют и другие среды разработки, такие как MonoDevelop и JetBrains Rider, Visual Studio остаётся лучшим выбором для разработки на Unity благодаря глубокой интеграции с игровым движком и широкой поддержке сообщества.

1.2.4 Информационное обеспечение

Для информационного обеспечения использовались:

- Microsoft Word – для создания документации для выпускной квалификационной работы, а так же для написания концепт-документа;
- Microsoft PowerPoint – для создания презентации выпускной квалификационной работы.

1.3 Обзор и анализ существующих программных систем

На данный момент нет аналогов, которые точно повторяют «Super Bomberman». Один из аналогов — это игра продолжение франшизы «Super Bomberman R». В этой версии добавили онлайн-режим, что позволяет играть через интернет. Но «Super Bomberman R» нельзя назвать полным аналогом, так как он отличается от классической, пиксельной 2D. Скриншот аналога изображён на рисунке 3.



Рисунок 3 – Скриншот игры «Super Bomberman R»

2 Специальная часть

2.1 Анализ задачи

2.1.1 Информационное моделирование предметной области

В рамках разработанной многопользовательской игры «Super Bomberman» выделены следующие основные сущности и связи между ними:

- игрок;
- сессия;
- раунд;
- карта;
- бомба;
- взрыв;
- бонусы.

Игрок является участником игровой сессии и характеризуется несколько параметрами. Для каждого игрока хранится его уникальный номер в комнате, текущее состояние, а также игровые характеристики:

- максимальное число одновременно установленных бомб;
- радиус взрыва;
- скорость передвижения.

Сессия соответствует комнате многопользовательской сети и задаёт общее количество раундов, которые должны быть сыграны, а также фиксирует количество уже завершённых раундов и счёт каждого участника. При подключении к комнате каждый новый игрок получает свой номер и добавляется в список активных участников.

Раунд запускается автоматически при начале игры или сразу после окончания предыдущего раунда. В начале каждого раунда всем игрокам присваивается состояние «в живых». Когда игрок погибает, это моментальное событие отмечается в его свойствах. После небольшой паузы ведущий клиент подсчитывает оставшихся в живых, обновляет счёт победителя данного раунда

и либо перезапускает уровень для следующего, либо завершает матч, если все запланированные раунды уже сыграны.

Карта представляет собой игровое поле, собранное из двух слоёв плиток:

- неразрушаемых стен;
- разрушаемых блоков.

При старте уровня сохраняется первоначальное расположение разрушаемых блоков, а во время игры сломанные блоки удаляются из игрового пространства. Кроме того, карта отслеживает, в каких именно клетках в настоящий момент находятся установленные бомбы, чтобы исключить возможность поставить две бомбы в одну и ту же ячейку.

Бомба создаётся в тот момент, когда игрок устанавливает её на текущей клетке. Объект бомбы содержит координату ячейки и ссылку на соответствующий скрипт управления. После установки бомба ждёт заранее заданное время активации, а затем взрывается. Контроллер бомбы вычисляет все клетки, в которые распространяется взрыв, удаляет саму бомбу из списка занятых ячеек, создаёт визуальные эффекты взрыва в нужных точках и разрушает блоки на карте в зоне поражения.

Взрыв состоит из трёх последовательных анимаций. Как только все три анимации проиграны, объект взрыва автоматически удаляется из сцены. Если во время взрыва он затрагивает игрока, запускается последовательность его уничтожения и дальнейшего завершения текущей игровой жизни.

При разрушении блоков с некоторой вероятностью появляются бонусы, среди которых могут быть дополнительная бомба, увеличение радиуса взрыва или ускорение. Если игрок проходит через зону действия бонуса, его игровые параметры сразу же обновляются: увеличивается максимально возможное количество одновременно установленных бомб, расширяется радиус взрыва или повышается скорость передвижения.

Связи между сущностями:

- каждый игрок может ставить несколько бомб в пределах одного раунда;

- каждая бомба при взрыве создаёт эффекты в четырёх направлениях до ближайшей стены или блока;
- игроки подбирают бонусы в процессе раунда, и эти бонусы изменяют их игровые параметры;
- все объекты бомб, взрывов и бонусов привязаны к ячейкам карты, что обеспечивает точное позиционирование и логику взаимодействия.

В данной реализации для хранения и передачи состояний используется внутренняя система свойств сетевой библиотеки, внешняя база данных не применяется.

ER диаграмма игры представлена на рисунке 4.

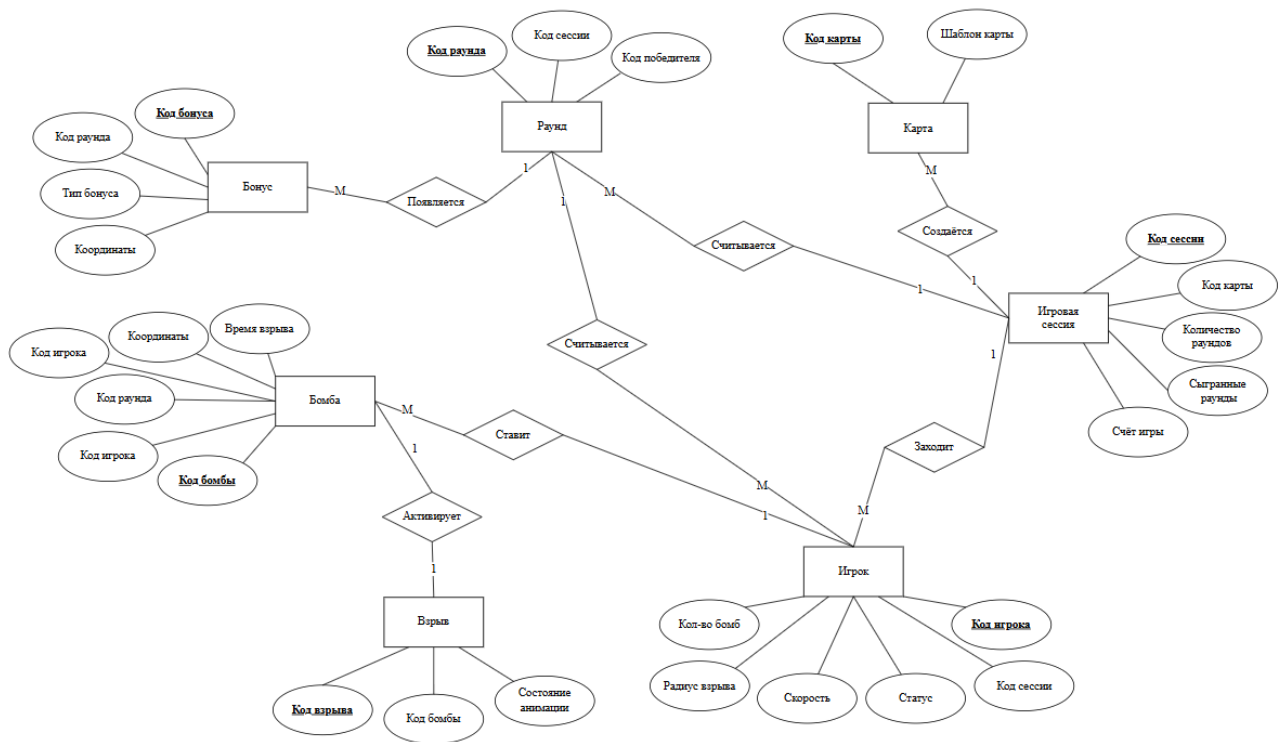


Рисунок 4 – ER диаграмма

2.1.2 Проектирование пользовательского интерфейса

Для проектирования и разработки прототипа интерфейса приложения была использована программа Balsamiq Mockups, которая позволяет создавать макеты с простым и интуитивно понятным дизайном. Прототип включает ключевые экраны, необходимые для полноценного взаимодействия пользователя с

приложением. Далее представлен краткий обзор этих экранов, а также обоснование расположения элементов на каждом из них.

Главное меню содержит основные элементы навигации, которые позволяют пользователю быстро перемещаться по разделам приложения. Центральное расположение кнопок навигации выбрано для упрощения доступа и обеспечения логичного потока действий — пользователь сразу видит, какие функции доступны, и может быстро перейти к интересующему разделу. Первый прототип главного меню изображён на рисунке 5.

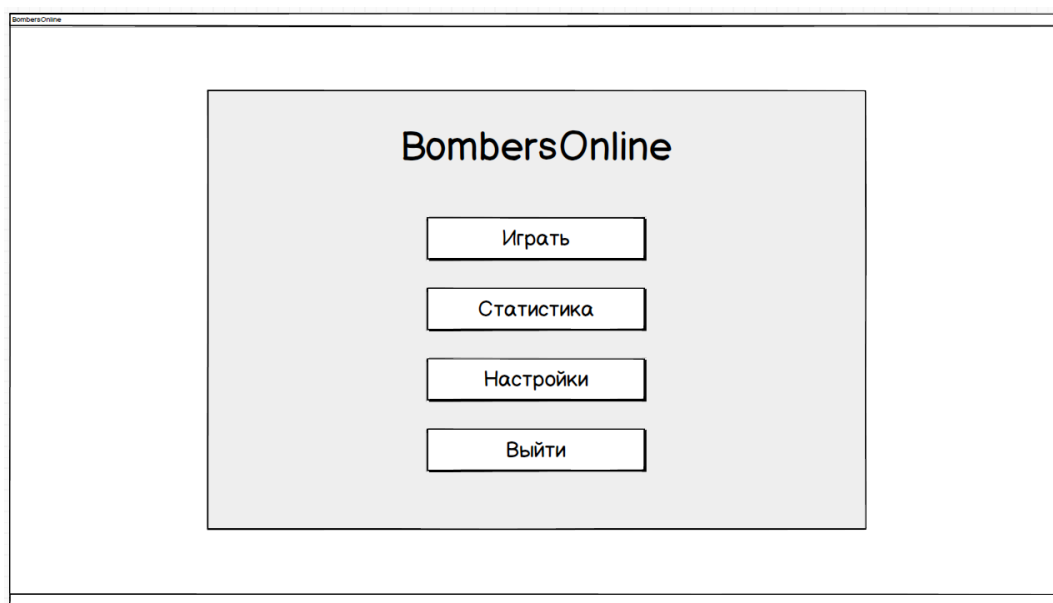


Рисунок 5 – Прототип главного меню

В разделе настроек игрок может изменить различные параметры игры, такие как громкость музыки, количество раундов и количество игроков. Параметры разделены на логические группы, что упрощает навигацию и делает процесс настройки интуитивно понятным. Например, элементы управления звуком расположены в одном блоке, а параметры, связанные с игровым процессом, — в другом, чтобы пользователь мог сразу найти нужную настройку. Первый прототип окна «настройки» изображён на рисунке 6.

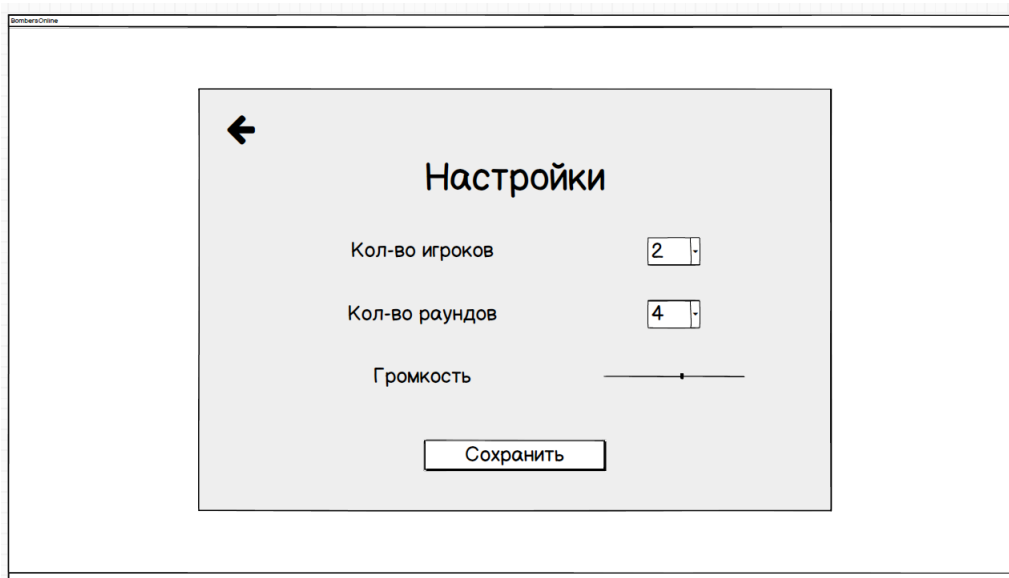


Рисунок 6 – Прототип меню настроек

Основной экран, на котором происходит игровой процесс. Элементы управления расположены в нижней части экрана для легкого доступа, а игровое поле занимает центральное положение, что помогает пользователю сосредоточиться на игровом процессе. Визуальные элементы и индикаторы, такие как очки и жизни, размещены на верхней панели, чтобы они всегда оставались в поле зрения игрока, не перекрывая основное игровое поле. Первый прототип основного экрана игры изображён на рисунке 7.



Рисунок 7 – Прототип окна игры

После завершения игрового процесса пользователь видит результаты игры и статистику по раундам. Результаты и информация по раундам расположены в центре экрана, чтобы игрок мог сразу получить обратную связь по своей игре. Кнопка «Выйти» расположена снизу, обеспечивая логичное завершение и возвращение к основным функциям. Первый прототип окна «окончания игры» изображён на рисунке 8.

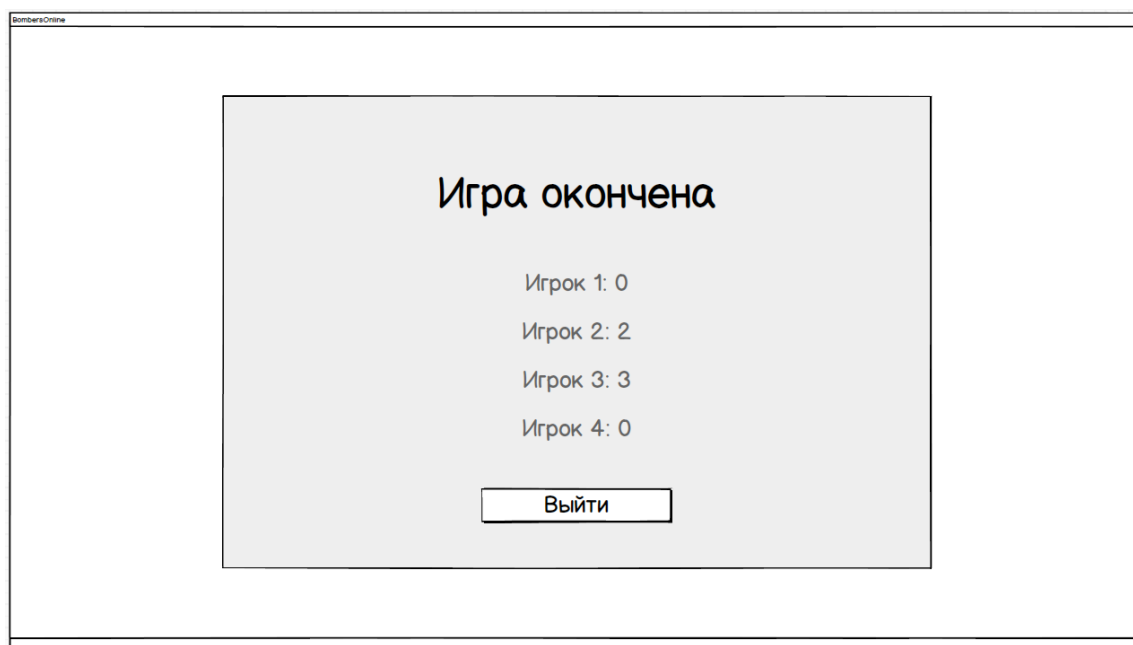


Рисунок 8 – Прототип окончания игры

Все элементы интерфейса были спроектированы с учетом удобства пользователя, чтобы обеспечить интуитивное управление и легкость в использовании. Ознакомиться с детальным дизайном экранов можно в приложении Б, на рисунках 1-8.

2.2 Описание логической структуры

2.2.1 Алгоритм программы

2.2.1.1 Алгоритмы игры

Здесь описываются алгоритмы основных механик игры, представленные ниже:

- алгоритм движения персонажа: каждый кадр контроллер перемещения считывает направление от клавиатуры или геймпада, рассчитывает цель в соседней клетке и проверяет наличие препятствий; если клетка свободна, персонаж плавно перемещается к её центру, иначе направление обнуляется;

- алгоритм размещения и взрыва бомб: по нажатию кнопки проверяется, свободна ли текущая клетка и остались ли доступные заряды бомб, создаётся объект бомбы с таймером, по истечении которого бомба удаляется, а запускается распространение взрыва;

- алгоритм расчёта области взрыва: в четырёх направлениях последовательно проверяются соседние клетки до заданного радиуса или до первой непреодолимой стены, все пройденные клетки добавляются в список поражения;

- алгоритм проигрыша и перезапуска раунда: при гибели персонажа его помечают как «мертвый», ведущий клиент ждёт краткую задержку, подсчитывает оставшихся в живых, обновляет счёт победителя текущего раунда и либо перезагружает уровень, либо завершает игру, если сыграны все раунды;

- алгоритм активации бонусов: после разрушения блока с заданной вероятностью появляется усиление, при соприкосновении с ним игрок немедленно получает увеличение числа бомб, радиуса взрыва или скорости движения;

- Алгоритм подсчёта очков: во время матча, с каждым новым раундом записывается победитель и ему прибавляются очки.

Блок-схема алгоритма размещения и взрыва бомб изображена на рисунке А.5, а код алгоритма находится в листинге Г.1.

Блок-схема алгоритм подсчёта очков изображена на рисунке А.7, а код алгоритма находится в листинге Г.3.

2.2.1.2 Алгоритмы мультиплеера

Здесь описываются алгоритмы сетевого взаимодействия, представленные ниже:

- алгоритм подключения к серверу и синхронизации сцены: клиент автоматически подключается к серверу при запуске, а ведущий клиент обеспечивает одновременную загрузку уровня у всех участников;
- алгоритм создания и присоединения к комнате: ведущий клиент задаёт параметры комнаты, другие участники подключаются по коду комнаты;
- алгоритм синхронизации состояний: позиции персонажей, состояние бомб и бонусов передаются через удалённые вызовы процедур, каждый клиент получает обновления о действиях остальных;
- алгоритм сетевого размещения и взрыва бомб: ведущий клиент создаёт сетевую копию бомбы и после её взрыва рассылает всем координаты поражённых клеток;
- алгоритм подсчёта и обновления очков: ведущий клиент обновляет счёт победителя и общее число сыгранных раундов в свойствах комнаты и рассылает изменения всем участникам;
- алгоритм обработки выхода игрока: при отключении участника его персонаж удаляется из сцены, и, если остается один игрок, ведущий клиент инициирует завершение матча.

Блок-схема алгоритма создания игровой комнаты изображена на рисунке А.6, код алгоритма в листинге Г.2.

2.2.1.3 Алгоритмы искусственного интеллекта

Здесь описываются алгоритмы поведения ботов, представленные ниже:

- алгоритм принятия решений ботом: после задержки реакции бот оценивает окружение, определяет ближайший предмет или угрозу и выбирает одно из состояний — патрулирование, поиск предмета, преследование, разрушение стены или эвакуация;

- алгоритм построения пути: для любой цели бот рассчитывает маршрут по сетке клеток с учётом препятствий и разрушаемых блоков методом A*;
- алгоритм обхода опасных зон: при обнаружении зоны будущего взрыва бот находит ближайшую клетку вне радиуса поражения и строит к ней безопасный маршрут;
- алгоритм постановки бомбы и эвакуации: бот ставит бомбу у цели, сохраняет время её взрыва и сразу после установки переходит к эвакуации по безопасному маршруту.

Каждый алгоритм был разработан с учетом многопользовательской игры, обеспечивая интересный игровой процесс, а также удобное взаимодействие с игрой.

2.2.2 Используемые методы

В процессе разработки программного комплекса применялись стандартные методы игрового движка Unity, функции сетевой библиотеки Photon PUN и собственные вспомогательные процедуры. Ниже приведены основные группы используемых методов.

Методы игровой механики:

- Awake вызывается в самом начале жизненного цикла объекта и используется для первоначальной инициализации компонентов: получения ссылок на коллайдеры, спрайт-рендереры и другие ключевые объекты сцены;
- Start срабатывает перед первым кадром и служит для регистрации объекта в менеджерах состояния и установки начальных параметров;
- Update выполняется каждый кадр и отвечает за обработку ввода пользователя: считывание нажатий клавиш или осей геймпада, расчет вектора движения и установка направления в компонент Rigidbody2D, а также за запуск сопрограмм для постановки бомб;
- FixedUpdate вызывается с постоянным шагом времени и обеспечивает корректную обработку физики, предотвращая «проскакивание» коллайдеров при быстром движении;

- `OnTriggerEnter2D` обрабатывает вход других коллайдеров в зону триггера, что позволяет реализовать подбор бонусов, получение урона от взрыва и запуск цепной реакции бомб;

- `OnTriggerExit2D` служит для отключения режима «триггера» у коллайдера после того, как все первоначально находившиеся в нём объекты покинули зону, и превращения бомбы в непреодолимое препятствие;

- `PlaceBomb` создает экземпляр бомбы на выбранной клетке, уменьшает счётчик доступных зарядов, регистрирует бомбу в менеджере карты и после таймера взрыва удаляет её, возвращая слот игроку;

- `ExplodeLocal` и `ExplodeNetwork` вычисляют область поражения по четырём направлениям, создают объекты взрывного эффекта, уничтожают разрушаемые блоки и обеспечивают восстановление слота бомбы;

- `CollectExplosionCells` рекурсивно проверяет соседние клетки, добавляя их в область взрыва до встречи с непреодолимым препятствием или достижения максимального радиуса.

Методы сетевой части:

- `PhotonNetwork.ConnectUsingSettings` инициализирует соединение с Photon-сервером на основании настроек проекта;

- `PhotonNetwork.CreateRoom` и `PhotonNetwork.JoinRoom` создают новую комнату или подключаются к существующей с заданными пользовательскими свойствами;

- `PhotonNetwork AutomaticallySyncScene` гарантирует одновременную загрузку сцены всеми участниками при начале нового раунда;

- `PhotonNetwork.Instantiate` инстанцирует сетевые объекты с присвоением уникального идентификатора `PhotonView` и распределением права владения между клиентами и мастером;

- `PhotonView.RPC` позволяет вызывать удалённые процедуры: уведомление о смерти игрока, инициация взрыва на стороне мастера, синхронное создание эффектов взрыва на всех клиентах;

– `Room.CustomProperties` и `CurrentRoom.SetCustomProperties` используются для обмена данными о результатах раунда и текущем счёте через свойства комнаты;

– `OnPlayerLeftRoom`, `OnMasterClientSwitched`, `OnRoomPropertiesUpdate` обрабатывают события отключения участников, смены мастера и обновления свойств комнаты для своевременного завершения конкурса или отображения финального экрана.

Методы искусственного интеллекта:

– `Decide` выбирает текущее состояние бота на основании расположения игроков, бонусов и ожидаемых взрывов;

– `FollowPath` получает маршрут из модуля поиска пути и перемещает бота к каждой следующей точке через движение по сетке с плавным переходом к центру клеток;

– `StartBombEvade` инициирует постановку бомбы для расчистки пути или отвлечения, рассчитывает время взрыва и запускает эвакуацию на безопасное расстояние;

– `EvadeTo` выбирает ближайшую свободную клетку за пределами зоны поражения и передаёт её в `FollowPath` для отхода;

– `IsInDanger` сканирует список ожидающих взрывов, сравнивает координаты потенциальных взрывов и текущее положение, чтобы заблаговременно переключиться в состояние эвакуации;

– `AddBomb` и `IncreaseExplosionRadius` корректируют характеристики бота при подборе соответствующих бонусов, синхронно с поведением игрока.

Каждый из перечисленных методов выполняет строго определённую задачу, взаимодействуя с менеджерами карты, физическим движком Unity и сетевыми вызовами Photon. Такой подход обеспечивает модульность, лёгкость сопровождения и расширения функционала программного комплекса.

2.2.3 Составные части программы и связи между ними

В состав игрового комплекса входят следующие основные модули:

- менеджер лобби: отвечает за создание и присоединение к комнатам, отображение списка доступных игроков, настройку параметров матча и переход в игровую сцену;
- менеджер игры: управляет жизненным циклом раунда, отслеживает состояние игроков, подсчитывает очки, обновляет пользовательский интерфейс и через Photon-свойства комнаты синхронизирует счёт и число сыгранных раундов у всех участников;
- менеджер карты: кэширует исходное состояние тайлов, регистрирует установленные бомбы, обрабатывает разрушение блоков и сбрасывает карту при старте нового раунда;
- контроллер передвижения: обрабатывает ввод от пользователя или бота, рассчитывает движение по клеточной сетке, проверяет столкновения с непреодолимыми объектами и плавно перемещает персонажа к центру свободной клетки;
- контроллер бомбы: реализует механики постановки бомб, запуска таймера взрыва, регистрации в менеджере карты, а также отправляет через RPC команду мастеру на создание взрыва;
- поведение бомбы: с помощью триггерных коллайдеров блокирует прохождение игроков, позволяет им свободно стоять на бомбе до выхода и не допускает постановки второй бомбы в той же клетке;
- компонент взрыва: отображает анимацию в трёх состояниях, уничтожает объекты и уведомляет контроллер передвижения о поражении игроков;
- система искусственного интеллекта: принимает решения о патрулировании, сборе бонусов, постановке бомб и эвакуации, рассчитывает маршрут через PathfindingGrid и взаимодействует с контроллерами движения и бомб;

- пользовательский интерфейс: включает экраны главного меню, настроек, лобби, игрового HUD и окон результата, при взаимодействии с которыми вызываются методы LobbyManager и GameManager.

Связи между модулями организованы через следующие механизмы:

- сетевая синхронизация: обеспечивается вызовами PhotonView.RPC для синхронного создания взрывов и перезагрузки уровней через PhotonNetwork.LoadLevel;

- обмен свойствами комнаты: GameManager записывает и считывает в CustomProperties текущие счёт и количество раундов, уведомляя всех клиентов об изменениях через OnRoomPropertiesUpdate;

- управление сценой: при завершении раунда GameManager инициирует перезагрузку текущей игровой сцены для всех участников, после чего MapManager восстанавливает исходное состояние уровня;

- взаимодействие через коллайдеры Unity: MovementController и BombBehaviour используют методы OnTriggerEnter2D и OnTriggerExit2D для обработки столкновений с бомбами, бонусами и зонами взрыва;

- взаимодействие между объектами и менеджерами: при спавне игроков GameManager создаёт сетевой префаб через PhotonNetwork.Instantiate и регистрирует его в списке активных участников, а MapManager предоставляет координаты клеток и их мировые позиции.

Взаимодействие между игровыми объектами и менеджерами. При появлении нового игрока GameManager инстанцирует префаб через PhotonNetwork.Instantiate и регистрирует его в списке активных участников. MapManager предоставляет координаты клеток и их мировые позиции, необходимые контроллерам для перемещения и размещения бомб.

Каждая подсистема чётко выполняет свою задачу, а связь между ними обеспечивается через сочетание событий Unity, прямых вызовов методов в менеджерах и сетевых процедур Photon PUN. Это архитектурное разделение облегчает поддержку, тестирование и последующее расширение функционала приложения. Архитектура изображена на рисунке А.2.

2.3 Описание работы программы

2.3.1 Общие сведения

Название программы: BombersOnline.

Краткое описание: Игра разработана в стиле популярной серии «Super Bomberman» и адаптирована под многопользовательский режим. В этой игре игроки управляют персонажами на ограниченной арене, расставляют бомбы, чтобы уничтожать препятствия и своих соперников. Программа содержит основные функции, необходимые для запуска матча, отображения результатов, настройки звука. Игра поддерживает онлайн-режим, что позволяет пользователям соревноваться друг с другом.

2.3.2 Функциональное назначение

Основное функциональное назначение программы — предоставление игрового процесса в режиме онлайн, где пользователи могут соревноваться друг с другом.

Программа включает в себя следующие основные функции:

- игровой процесс: основная сцена, на которой игроки размещают бомбы, уничтожают блоки и соперников. каждая бомба имеет определенный радиус взрыва, который можно увеличить через бонусы;
- система улучшений: игроки могут собирать бонусы, которые улучшают их персонажа, также через меню магазина доступны дополнительные улучшения;
- настройки: меню, позволяющее настроить громкость звуковых эффектов, музыки, а также другие параметры;
- режим перезапуска и выхода: после завершения матча игрокам предлагается перезапустить уровень или выйти в главное меню.

2.3.3 Входные данные

При запуске и во время работы программа получает:

- данные игрока: уникальный ActorNumber в Photon при онлайн-подключении;

- параметры комнаты: желаемое число игроков от 2 до 4, количество раундов, режим «с ботами» или «только люди»;
- настройки пользователя: уровни громкости музыки;
- сетевые события: пакеты от Photon о присоединении/отключении игроков, RPC-вызовы смерти, взрывов и перезапуска уровней;
- игровой ввод: нажатия клавиш для передвижения по сетке и установки бомб.

2.3.4 Выходные данные

Программа отображает и передаёт:

- графический вывод: трёхслойная анимация взрыва, спрайты персонажей, тайлы уровня и интерфейсные элементы в реальном времени;
- аудиоинформацию: звуковые эффекты, фоновую музыку;
- уведомления и сообщения: предупреждения об ошибках подключения, результаты раунда и итоги матча;
- сетевой выход: обновлённые свойства комнаты, синхронизированные через `customproperties` и `rpc`, чтобы все клиенты отображали одно и то же состояние игры;
- логи и статистику: внутренняя запись событий для отладки и последующего анализа качества соединения и производительности.

2.3.6 Тестирование

В процессе разработки приложения BombersOnline были использованы различные методы тестирования, обеспечивающие высокое качество и надежность игрового процесса:

- Unit-тесты: модульные тесты были проведены для проверки игровых механик. Это позволило выявить и устранить ошибки на ранних стадиях разработки, обеспечивая надежность игрового процесса;
- тестирование по тестовым сценариям: для более полного и систематического тестирования игровых возможностей были разработаны сценарии, описывающие различные боевые ситуации и взаимодействия в игре.

Благодаря комплексному подходу к тестированию удалось обеспечить высокое качество при использовании приложения BombersOnline.

С тестами можно ознакомиться в приложении В, на рисунках В.1-6.

2.3.7 Вызов и загрузка

Запуск в браузере – игра собрана в формате WebGL и размещается на веб-сервере. Пользователь открывает в браузере URL-адрес приложения, после чего выполняется загрузка всех необходимых файлов: HTML-обёртки, JavaScript-движка Unity и ассетов. По завершении загрузки движок инициализируется, загружается стартовая сцена, и игрок попадает на главный экран игры.

2.4 Руководство оператора

2.4.1 Назначение программы

Web-игра BombersOnline создана для организации соревновательных матчей в духе классической «Super Bomberman» и рассчитана на динамичное взаимодействие нескольких игроков в реальном времени.

Область применения:

- для игроков: позволяет быстро и удобно создавать или присоединяться к игровым комнатам, выбирать персонажей и карты, настраивать количество раундов и звуковые параметры, отслеживать собственную статистику и соревноваться с другими участниками;
- для разработчиков и техподдержки: обеспечивает сбор логов и метрик производительности, что упрощает диагностику и исправление ошибок, а также позволяет внедрять обновления игровых механик и балансировать карту без необходимости выпуска новой версии клиента.

2.4.2 Условия выполнения

Для запуска веб-версии игры BombersOnline требуется:

- веб-браузер с поддержкой WebGL последней версии, установленный на устройстве;

- низкая пинг-латентность, не более 100 мс до сервера Photon PUN для обеспечения синхронизации состояний в реальном времени;
- стабильное интернет-соединение с пропускной способностью не менее 1 Мбит/с для обеспечения бесперебойного обмена данными в режиме реального времени.

Установка: для использования веб-версии игры пользователям необходимо иметь установленный современный веб-браузер, поддерживающий технологию WebGL. После открытия игры через браузер пользователи могут сразу приступить к игровому процессу без необходимости установки дополнительного программного обеспечения. Это обеспечивает максимальную доступность игры для широкой аудитории и позволяет играть на различных персональных компьютерах, поддерживающих указанные браузеры.

2.4.3 Выполнение программы

2.4.3.1 Работа пользователя

Игрок в браузерной версии BombersOnline последовательно выполняет следующие действия:

- создание или присоединение к комнате: на экране лобби пользователь может создать новую игровую комнату — задать число участников и раундов — или ввести код существующей, чтобы присоединиться к ней;
- настройка параметров матча: после входа в комнату любой участник может менять настройки — выбирать карту из доступных, устанавливать длительность раунда и количество раундов, а также настраивать громкость звуков и музыки;
- внутренний запуск матча: по нажатию кнопки «Старт» мастер-клиент автоматически инициирует загрузку игровой сцены и размещает каждого игрока на своей стартовой позиции;
- игровой процесс: в ходе раунда игрок управляет своим персонажем, перемещается по сетке, размещает бомбы, собирает выпавшие бонусы и старается остаться единственным выжившим;

- завершение раунда: по смерти всех, кроме одного участника, или истечении тайм-аута раунд считается завершённым — на экране показываются результаты;

- завершение матча: после последнего раунда выводится итоговая таблица всех игроков с количеством побед и поражений, а затем предлагается вернуться в лобби или выйти в главное меню.

2.4.3.2 Работа администратора комнаты

Пользователь, который создал комнату, дополнительно выполняет следующие задачи:

- синхронизация сцены: загружает и синхронизирует игровую сцену для всех участников при старте каждого раунда;

- управление стартом раунда: контролирует момент начала и окончания раунда, отправляя соответствующие сетевые события другим клиентам;

- сохранение и рассылка счётов: после каждого раунда обновлённые результаты записываются в свойства комнаты и рассылаются всем участникам для отображения на их интерфейсе;

- обработка выхода участников: при отключении или выходе игрока мастер-клиент проверяет количество оставшихся живых участников и при необходимости завершает матч досрочно, объявляя победителем единственного оставшегося.

2.4.4 Техника безопасности при работе на компьютере

2.4.4.1 Оборудование и организация рабочего места

Для обеспечения безопасной работы с компьютером крайне важно правильно организовать рабочее место. Это позволит не только повысить производительность, но и избежать возможных травм и заболеваний.

Рабочее место должно быть обустроено с учетом эргономических стандартов. Сиденье должно обеспечивать хорошую поддержку спины, а стол и монитор — быть на таком уровне, чтобы сотрудник мог работать, не напрягаясь. Экран должен располагаться на уровне глаз, а кресло должно поддерживать правильную осанку, позволяя ногам стоять на полу или на подставке.

Освещенность рабочего места также играет важную роль. Следует избегать резких контрастов между экраном и окружающим освещением, чтобы минимизировать нагрузку на глаза. Рекомендуется использовать настольные лампы с регулируемой яркостью и избегать прямых солнечных лучей, попадающих на экран.

Эргономика — важнейшая составляющая безопасности. При длительной работе необходимо соблюдать правильные позы, не сутулиться и регулярно менять положение тела для предотвращения усталости и болезней опорно-двигательного аппарата.

2.4.4.2 Защита здоровья и глаз

Работа за компьютером может привести к напряжению глаз и других проблем со здоровьем, если не соблюдать регулярные перерывы и правильную настройку оборудования.

Перерывы: Работникам рекомендуется делать краткие перерывы каждые 20-30 минут работы за компьютером. Во время перерыва желательно встать, размяться и посмотреть на объекты, расположенные на расстоянии не менее 6 метров. Это помогает снизить напряжение глаз и улучшить кровообращение.

Настройка экрана: Яркость и контрастность экрана должны быть настроены таким образом, чтобы они не вызывали напряжения глаз. Экран не должен быть слишком ярким или тусклым по отношению к освещению в помещении.

2.4.4.3 Защита данных и информационная безопасность

Для предотвращения угроз безопасности важно соблюдать требования по защите данных и безопасности работы с компьютерной техникой.

Антивирусная защита: Все устройства должны быть оснащены актуальными антивирусными программами. Регулярное обновление этих программ является обязательным для защиты от вирусов, вредоносных программ и других угроз.

Парольная защита: для защиты от несанкционированного доступа сотрудники должны использовать сложные пароли для учетных записей. Пароли должны быть уникальными для каждого сервиса и периодически обновляться.

Безопасность при загрузке данных: важно загружать программное обеспечение и файлы только из проверенных и надежных источников. Избегайте установки программ с сомнительных сайтов, чтобы предотвратить загрузку вредоносных программ.

2.4.4.4 Правила работы с периферийными устройствами

Корректное использование периферийных устройств также важно для обеспечения безопасности и предотвращения несчастных случаев.

Использование мыши и клавиатуры: Долговременная работа с мышью и клавиатурой должна происходить с соблюдением правильной осанки. Рекомендуется использовать устройства, соответствующие размеру руки, и избегать перенапряжения.

Предотвращение перегрева устройств: Компьютеры и другие устройства не должны использоваться в условиях перегрева. Важно следить за тем, чтобы вентиляционные отверстия на устройствах не были заблокированы. При работе с ноутбуками рекомендуется использовать охлаждающие подставки.

3 Экономическая часть

3.1 Расчёт заработной платы с отчислениями на социальное страхование

Смета затрат на производство — это общий свод плановых затрат на производство продукции, выполнение работ и услуг в соответствии с производственной программой предприятия. Расчет сметы затрат на производство осуществляется на основе бизнес-плана в части производственной программы на будущий финансовый год, а также анализа фактических данных прошедшего периода и расчетных нормативов по статьям затрат, принятых в организации.

Расчетные нормативы по статьям затрат планируются на основе анализа действующих рыночных цен на аналогичную выпускаемую продукцию, фактических цен реализации продукции прошедшего финансового года.

По этим данным определяется предельный норматив затрат на производство при существующем уровне рыночной цены. Рассчитанный таким образом предельный норматив затрат на единицу продукции с учетом принятых на предприятии нормативов по статьям затрат позволяет определить плановую смету в разрезе статей и элементов.

С учетом специфики отраслевой деятельности организации составление сметы затрат на производство ведется по видам деятельности, видам продукции, работ, услуг; дальнейшая группировка возможна по структурным подразделениям.

На заключительном этапе планирования сметы затрат на производство составляется сводная смета под плановую производственную программу с учетом объемов выпуска.

На первом этапе разработки программного продукта необходимо рассчитать смету затрат, а также определить этапы разработки программного продукта и трудоёмкость выполнения каждого этапа. Все показатели представлены в таблице 1.

Таблица 1 — Этапы и трудоёмкость выполнения работ в часах

Этапы разработки	Руководитель	Техник
Постановка задачи	2	12
Выбор инструментальных средств	1	3
Разработка мат. модели	0	14
Построение алгоритма	2	18
Программирование	0	52
Тестирование	8	32
Отладка	4	24
Написание пояснительной записки	3	16
Итого	20	171

Минимальный размер заработной платы на предприятии не может быть ниже минимального размера заработной платы.

Месячный оклад руководителя составляет 35000 руб., месячный оклад техника составляет 25000 руб.

3.1.1 Расчёт основной заработной платы

При расчёте основной заработной платы учитывается заработная плата всех категорий работников, непосредственно занятых разработкой программного продукта. Размер заработной платы определяется исходя из количества исполнителей и их квалификационного уровня, а также затраченного ими времени в целом на разработку программного продукта и отдельных его этапов.

Основная заработная плата работников определяется по повременной системе оплаты труда.

Рассчитаем цену одного часа работы сотрудника по формуле 1:

$$Ц_{\text{ч}} = O_{\text{м}} / (D_{\text{р}} \times Ч), \quad (1)$$

где $O_{\text{м}}$ — месячный оклад;

$D_{\text{р}}$ — среднее число рабочих дней в месяце;

$Ч$ — часов в смене.

В соответствии с описанными ранее окладами и числом рабочих дней в апреле равным 22 произведем расчёт оплаты труда за час.

Руководитель проекта:

$$\text{Ц}_\text{ч} = 35000 / (22 \times 8) = 198,86 \text{ руб.}$$

Техник:

$$\text{Ц}_\text{ч} = 25000 / (22 \times 8) = 142,04 \text{ руб.}$$

Заработная плата работников по окладу рассчитывается по формуле 2:

$$\text{ЗП}_\text{осн} = \text{Ц}_\text{ч} \times \text{Ч}, \quad (2)$$

где Ч — количество часов, затраченных на разработку программного продукта.

Таблица 2 — Основная заработная плата работников

В рублях

Должность	Количество часов, затраченных на разработку продукта, час	Оплата труда за 1 час	Заработная плата
Руководитель проекта	20	198,86	3977,2
Техник	171	142,04	24288,84
Итого			28266,04

Таким образом, по показателям, представленным в таблице 2, основная заработная плата работников составила 28266,04 руб.

3.1.2 Расчет дополнительной заработной платы

Дополнительная заработная плата включает различные виды доплат сверх основной заработной платы: премии, доплату за работу в сверхурочное время, надбавки за профессиональное мастерство, оплату очередного и учебных отпусков и прочие виды доплат.

Дополнительная заработная плата устанавливается на предприятии в процентах от суммы основной заработной платы и составляет от 16 до 20%.

Таким образом, дополнительная заработная плата будет определяться по следующей формуле:

$$ЗП_{\text{доп}} = ЗП_{\text{осн}} \times 20\% / 100\%, \quad (3)$$

где $ЗП_{\text{осн}}$ — основная заработная плата, руб.

Значение $ЗП_{\text{осн}}$ возьмём из таблицы 2, и рассчитаем дополнительную заработную плату.

Значение $ЗП_{\text{осн}}$ возьмём из таблицы 2, и рассчитаем дополнительную заработную плату.

$$ЗП_{\text{доп}} = 28266,04 \times 0,2 = 5653,21 \text{ руб.}$$

На основании вышеприведенных расчетов, делаем вывод, что основная и дополнительно заработная плата работников по разработке мобильного приложения расписания Политехнического колледжа НовГУ составит $28266,04 + 5653,21 = 33919,25$ руб.

3.1.3 Расчёт отчислений на социальное страхование

Отчисления на социальное страхование составляют 30% от суммы основной и дополнительной заработной платы. Отчисления на социальное страхование рассчитываются по формуле 4[3]:

$$З_{\text{стр}} = (ЗП_{\text{осн}} + ЗП_{\text{доп}}) \times 30\% / 100\%, \quad (4)$$

где $ЗП_{\text{осн}}$ — основная заработная плата, руб.;

$ЗП_{\text{доп}}$ — дополнительная заработная плата, руб.

Значение $ЗП_{\text{осн}}$ возьмём из таблицы 2, а $ЗП_{\text{доп}}$ рассчитывается по формуле 3. Вычислим величину отчислений с помощью формулы 4.

$$З_{\text{стр}} = (33919,25) \times 0,3 = 10175,78 \text{ руб.}$$

Таким образом, из расчетов видно, что отчисления на социальное страхование составят 10175,78 руб.

3.2 Расчёт стоимости материалов и лицензионного обеспечения

При создании программного продукта используется лицензионное программное обеспечение, поэтому необходимо включить стоимость соответствующих программ в смету затрат.

Определим перечень лицензионных программ и норму установки в таблице 3.

Таблица 3 — Перечень программного обеспечения для выполнения работ

Наименование лицензионных программ	Норма установки, шт.	Цена, руб.
Unity	1	0
Visual Studio	1	0
Aseprite	1	435
Balsamiq Mockups	1	1200
Microsoft Office	1	5199
Итого		6834

Стоимость лицензионного программного обеспечения для разработки программного продукта составит 6834 руб.

Расчёт стоимости необходимых материалов для разработки программного продукта рассчитывается на основе норм расхода материальных ресурсов и оптовых цен на их приобретение по формуле 5:

$$C_m = N_p \times Ц, \quad (5)$$

где N_p — норма расхода материальных ресурсов в натуральных единицах;

$Ц$ — цена приобретения за единицу, руб.

Расчёт стоимости материалов представлен в таблице 4.

Таблица 4 — Расчёт стоимости материалов

Наименование материала	Норма расхода	Цена за единицу, руб	Сумма, руб
Бумага, уп.	1	450	450
Картридж, шт.	1	2000	2000
Папка, шт.	1	250	250
Итого			2700

Общая стоимость материалов составит 2700 руб.

Итого прямых расходов — $33919,25 + 10175,78 + 6834 + 2700 = 53629,03$ руб.

3.3 Расчёт накладных расходов

Накладные расходы представляют собой дополнительные к основным расходам затраты на управление, организацию и обслуживание производства.

Не связаны напрямую с основным производством товаров или предоставлением услуг, не входят в стоимость материалов и оплату труда.

Накладные расходы закладываются в себестоимость товара, издержки его производства и обращения, но не прямо, а косвенно, пропорционально стоимости материалов и сырья, сумме заработной платы и так далее.

Накладные расходы составляют 10% от расходов на эксплуатацию и рассчитываются по формуле 6:

$$H_p = (P_{\text{экс}} \times 10\%) / 100\%,$$

где $P_{\text{экс}}$ — величина расходов на эксплуатацию, руб.

Принимаем, что $P_{\text{экс}} = 53629,03$ руб.

$$H_p = (53629,03 \times 10\%) / 100\% = 5362,90 \text{ руб.}$$

На основании выше рассчитанных данных составим смету затрат на программный продукт в таблице 5.

Таблица 5 — Смета затрат на программный продукт

Статьи затрат	Сумма, руб.	Структура затрат, %
Основная и дополнительная заработная плата	33919,25	57,5
Отчисления на социальное страхование	10175,78	17,25
Стоимость материалов	6834	11,58
Стоимость лицензионного обеспечения	2700	4,58
Итог расходы на эксплуатацию	53629,03	90,91
Накладные расходы	5362,90	9,09
Итого	58991,93	100

Таким образом, общая сумма затрат на разработку составит 58991,93 руб.

По результатам расчета построим диаграмму структуры затрат на программный продукт и представим её на рисунке 7.



Рисунок 9 – Структура затрат на программный продукт

3.4 Составление и расчёт цены реализации программного продукта

Цена является одним из самых важных экономических показателей. Её основная функция состоит в обеспечении выручки от реализации программного продукта, поэтому цена определяет прибыль и финансовую стабильность предприятия, его жизнеспособность.

Так как целью предпринимательской деятельности является получение прибыли, то цена реализации программного продукта будет представлять собой сумму затрат на его разработку и запланированную величину прибыли. Примем уровень рентабельности $P = 30\%$. Величину общих затрат на разработку программного продукта возьмём из таблицы 5, а стоимости материалов из таблицы 4.

Расчёт оптовой цены производится по формуле 7[4]:

$$Ц_{\text{опт}} = З \times (1 + P),$$

где $З$ — общие затраты на разработку программного продукта, руб.;

P — уровень рентабельности проекта, коэффициент.

Произведём расчет оптовой цены используя формулу 7

Принимаем $З = 58991,93$.

$$Ц_{\text{опт}} = 58991,93 \times (1 + 0,30) = 76689,51 \text{ руб.}$$

Расчёт прибыли от продажи продукта производится по формуле 8:

$$\Pi = Ц_{\text{опт}} - З,$$

где $Ц_{\text{опт}}$ — оптовая цена, руб.;

$З$ — общие затраты на разработку программного продукта, руб.

Принимаем $Ц_{\text{опт}} = 76689,51 - 58991,93$.

$$\Pi = 76689,51 - 58991,93 = 17697,58 \text{ руб.}$$

Таким образом, можно сделать вывод, что разрабатываемый продукт прибыльный и может быть реализован. Прибыль от продажи программного продукта составит 17697,58 руб.

Заключение

В рамках выпускного проекта разработана браузерная многопользовательская игра BombersOnline в стиле классического «Super Bomberman» с встроенным искусственным интеллектом. Были реализованы ключевые механики: динамичное движение по сетке, постановка и взрыв бомб, сбор бонусов, а также сетевое взаимодействие через Photon PUN и поведение ботов с поиском пути.

При проектировании выбраны Unity и Photon благодаря их кроссплатформенности и возможностям синхронизации. Тестирование подтвердило стабильность, а система сжимаемых RPC-сообщений и кэширование тайлов обеспечили плавный геймплей и низкую нагрузку на сеть.

Итоговый продукт готов к публикации и дальнейшему расширению: добавлению новых карт, режимов и усовершенствованию искусственного интеллекта противников.

Список литературы

1. ГОСТ 2.701—2008. Единая система конструкторской документации. Схемы. Виды и типы. Общие требования к выполнению. — Москва: Стандартинформ, 2009. — 15 с.
2. Чернега А. М. Методические рекомендации по организации выполнения и защиты выпускной квалификационной работы: указания для студентов. — Великий Новгород: ПТК НовГУ, 2011. — 52 с.
3. Пример расчета страховых взносов на обязательное страхование в 2024 году [Электронный ресурс]. — Режим доступа: <https://www.kontur-extern.ru/info/raschet-strahovyh-vznosov-v-pfr/>. — Дата обращения: 01.06.2025.
4. Елена Руднева Оптовая цена [Электронный ресурс]. — Режим доступа: https://www.banki.ru/wikibank/optovaya_cena/. — Дата обращения: 01.06.2025.
5. Официальная документация Unity [Электронный ресурс]. — Режим доступа: <https://docs.unity3d.com/>. — Дата обращения: 27.03.2025.
6. Официальная документация PhotonPun [Электронный ресурс]. — Режим доступа: <https://doc.photonengine.com/pun/>. — Дата обращения: 24.05.2025.
7. Страница публикации игры [Электронный ресурс]. — Режим доступа: <https://brikka.itch.io/bombersonline/>. — Дата обращения: 31.05.2025.
8. Страница исходного кода игры [Электронный ресурс]. — Режим доступа: <https://github.com/Brikka23/BombersOnline/>. — Дата обращения: 17.05.2025.

Приложение А
(обязательное)

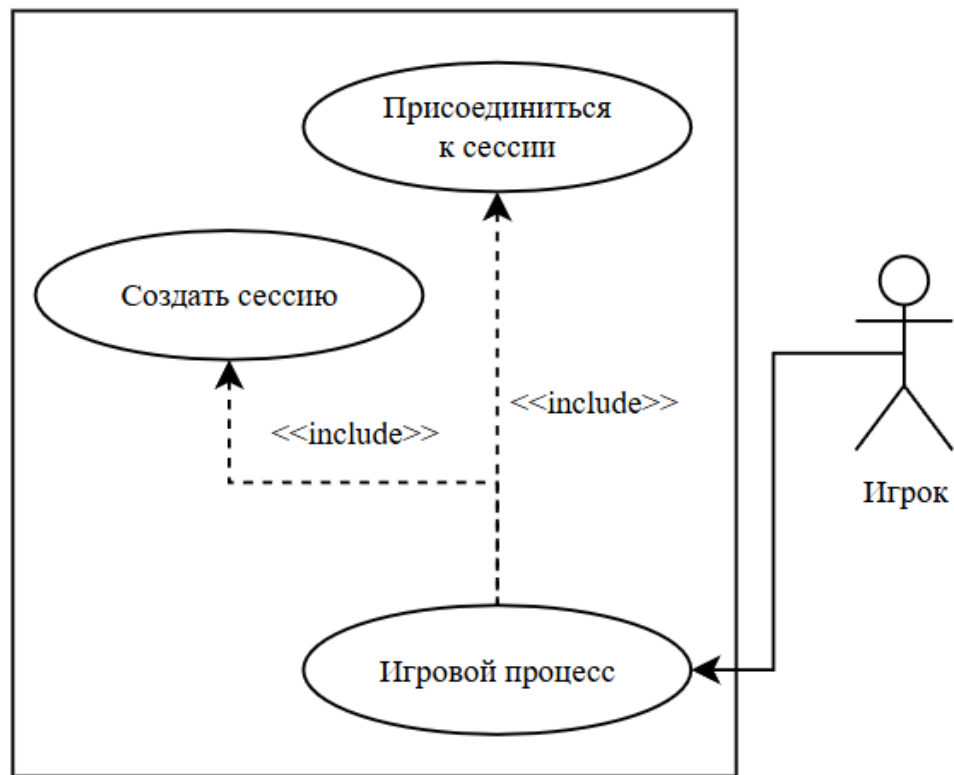


Рисунок А.1 – Диаграмма прецедентов

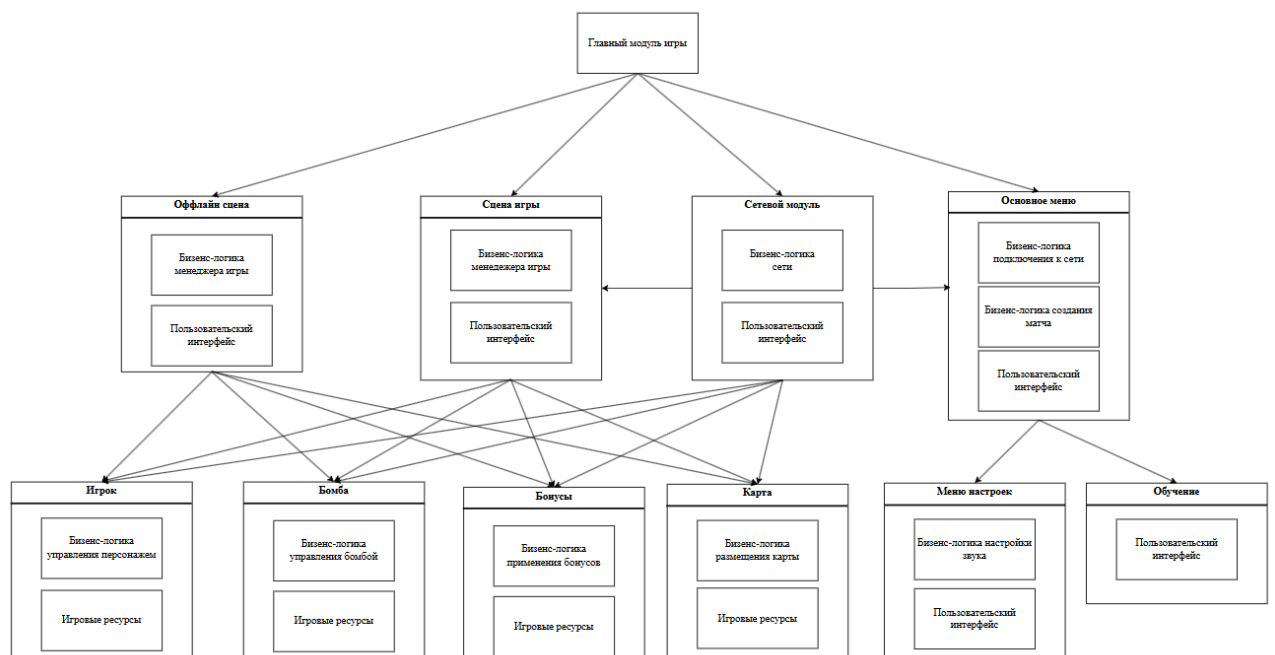


Рисунок А.2 – Диаграмма архитектуры программного комплекса



Рисунок А.3 – Диаграмма взаимодействия программных моделей

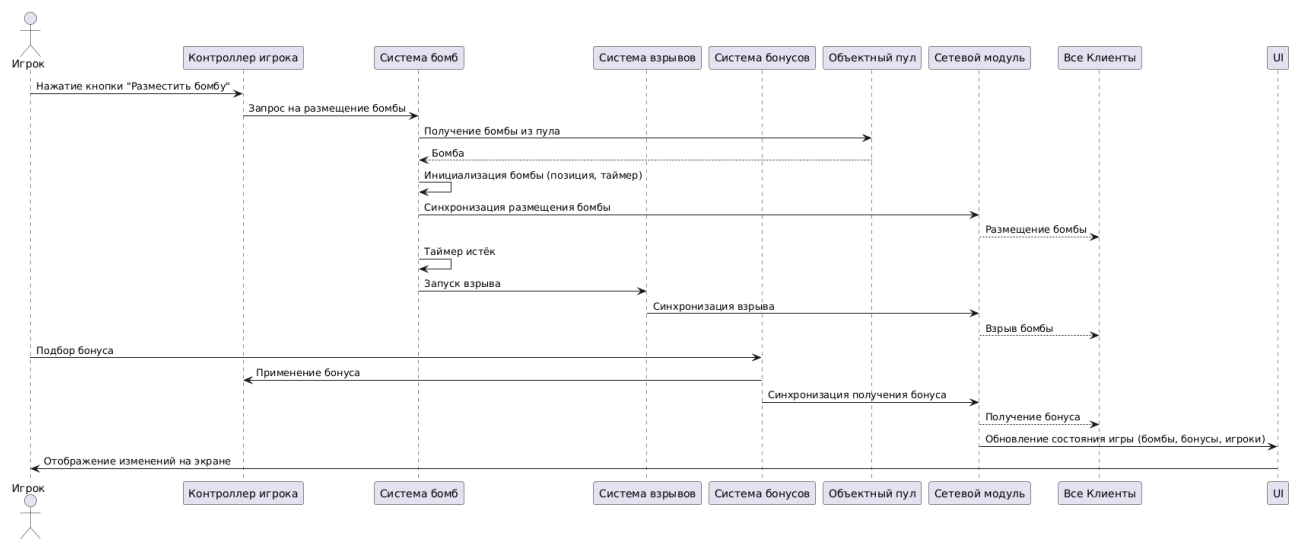


Рисунок А.4 – Диаграмма взаимодействия внутри модуля матча

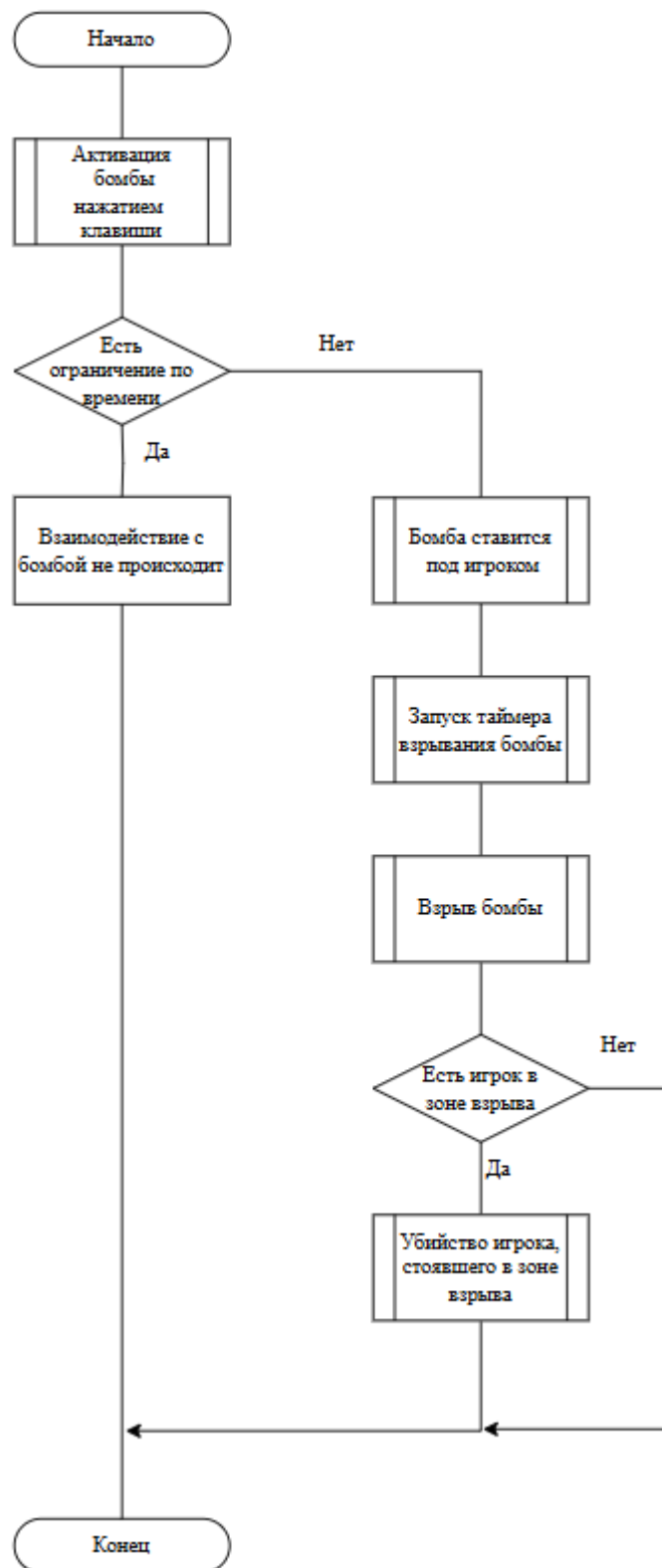


Рисунок А.5 – Алгоритм взаимодействия с бомбой

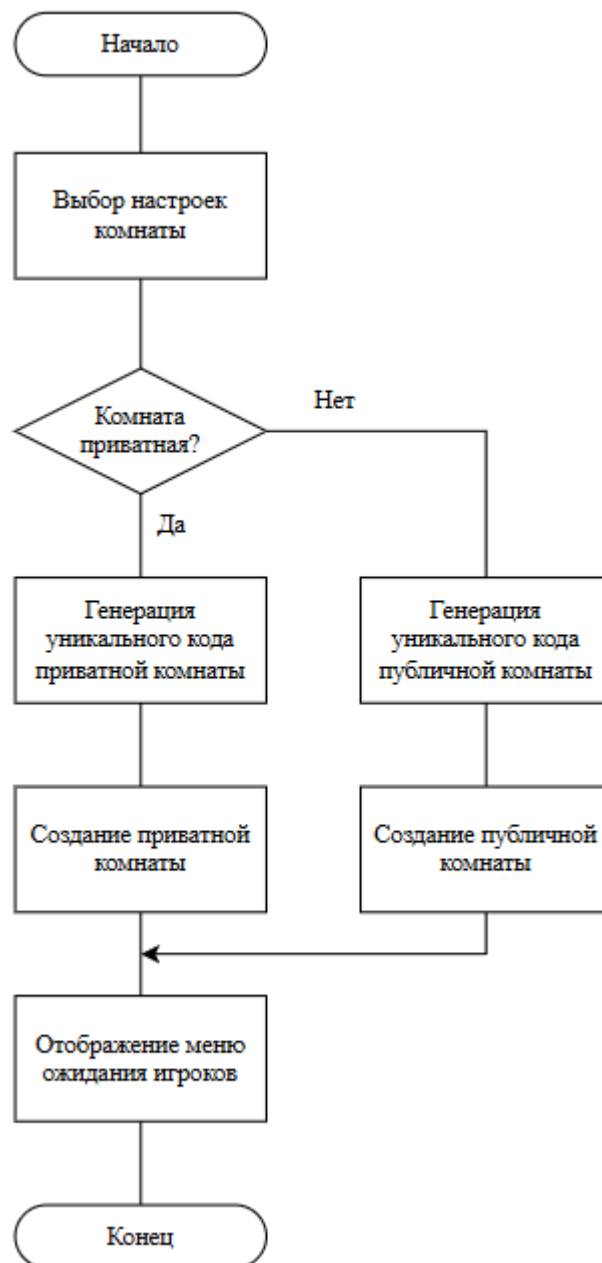


Рисунок А.6 – Алгоритм создания игровой комнаты

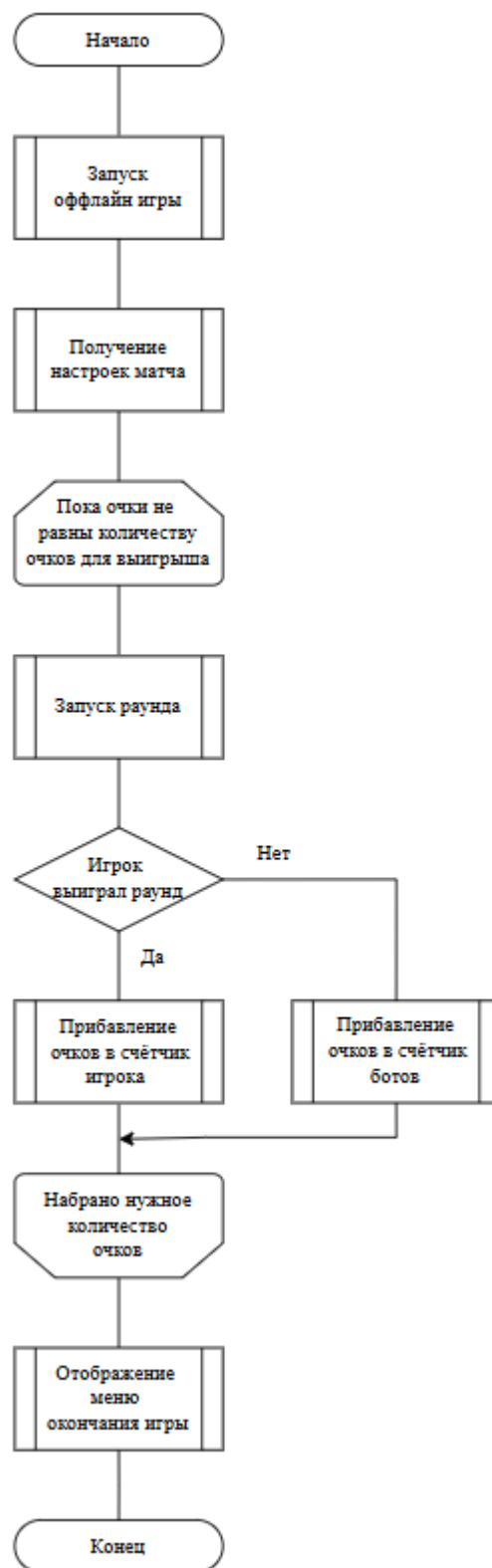


Рисунок А.7 – Алгоритм подсчёта очков в офлайн режиме

Приложение Б
(обязательное)

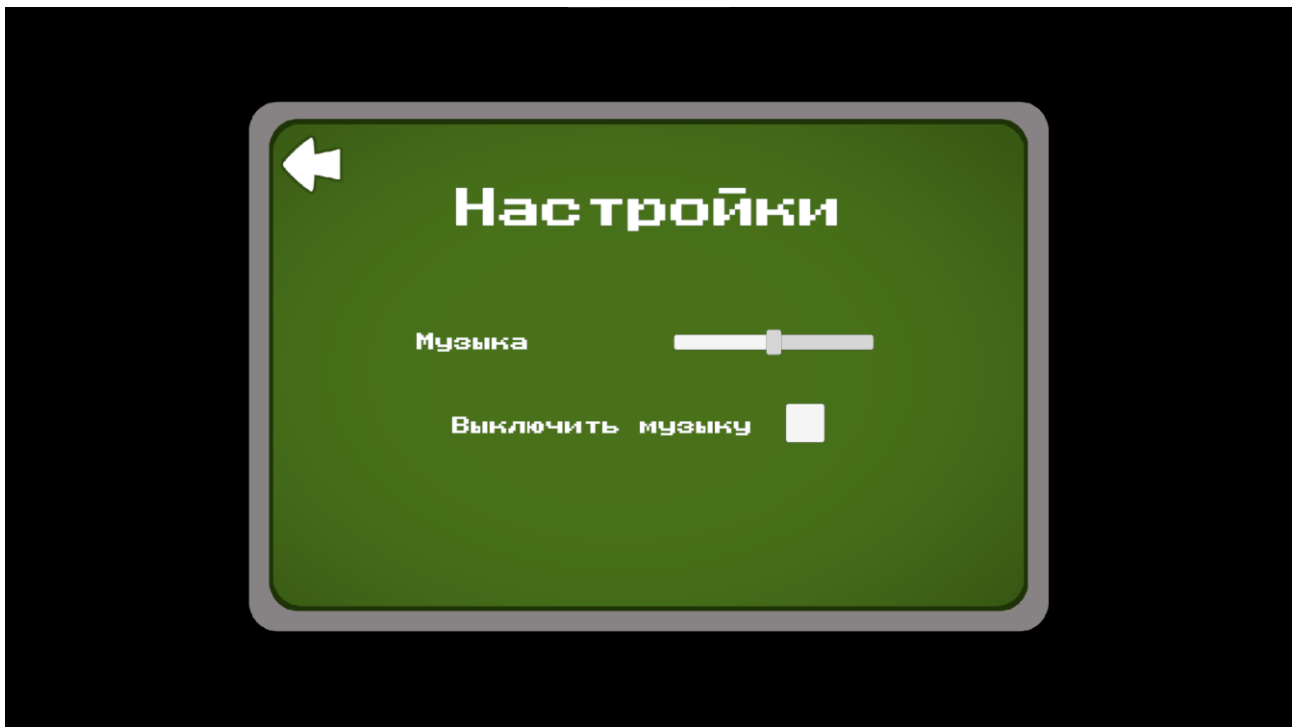


Рисунок Б.1 – Меню настроек игры



Рисунок Б.2 – Главное меню

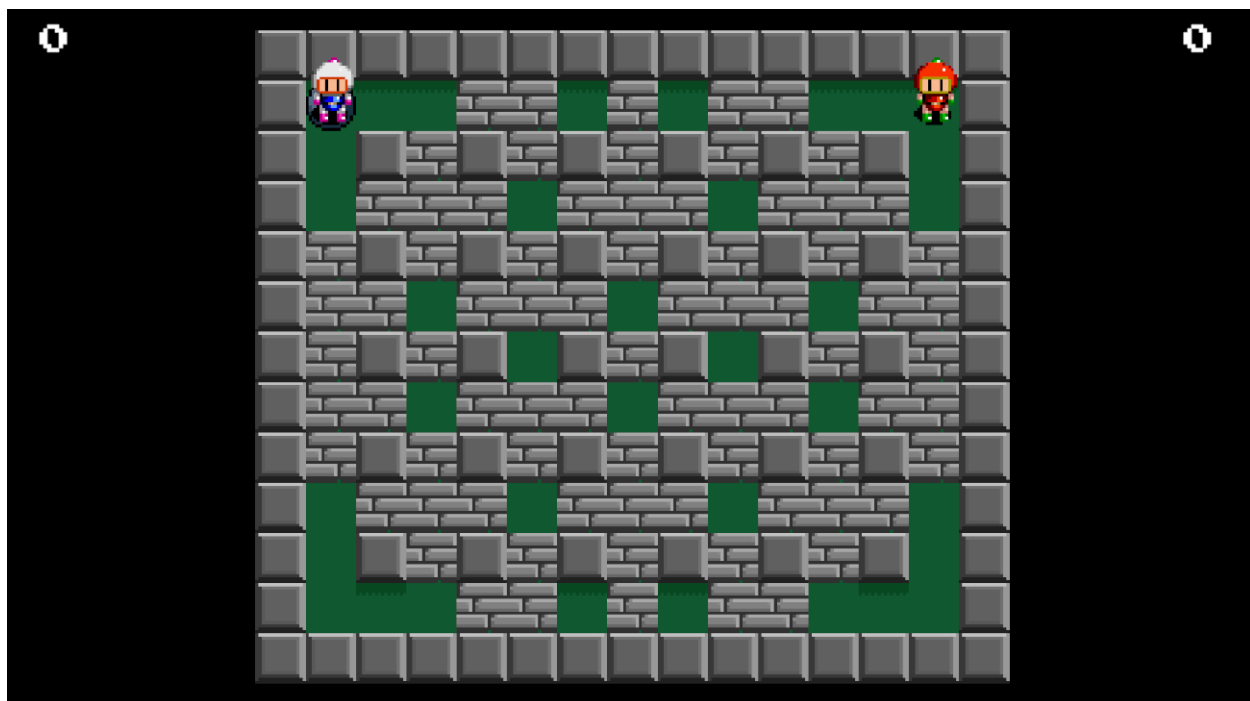


Рисунок Б.3 – Окно игры

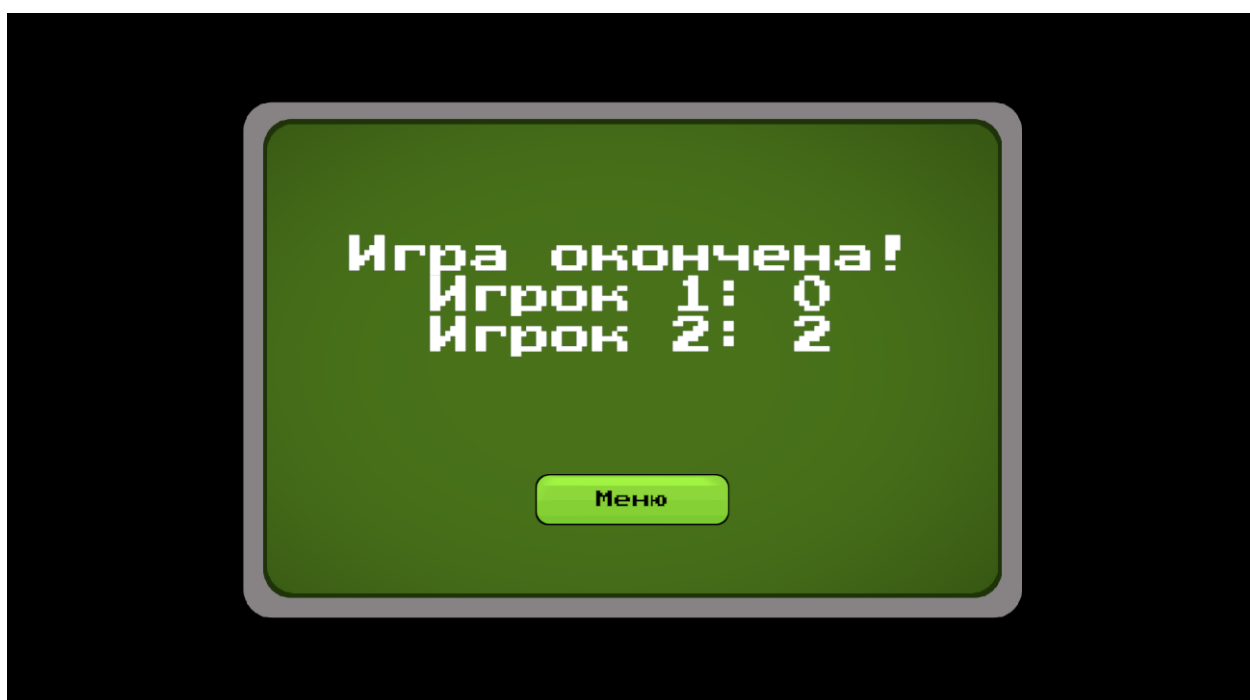


Рисунок Б.4 – Окно окончания игры

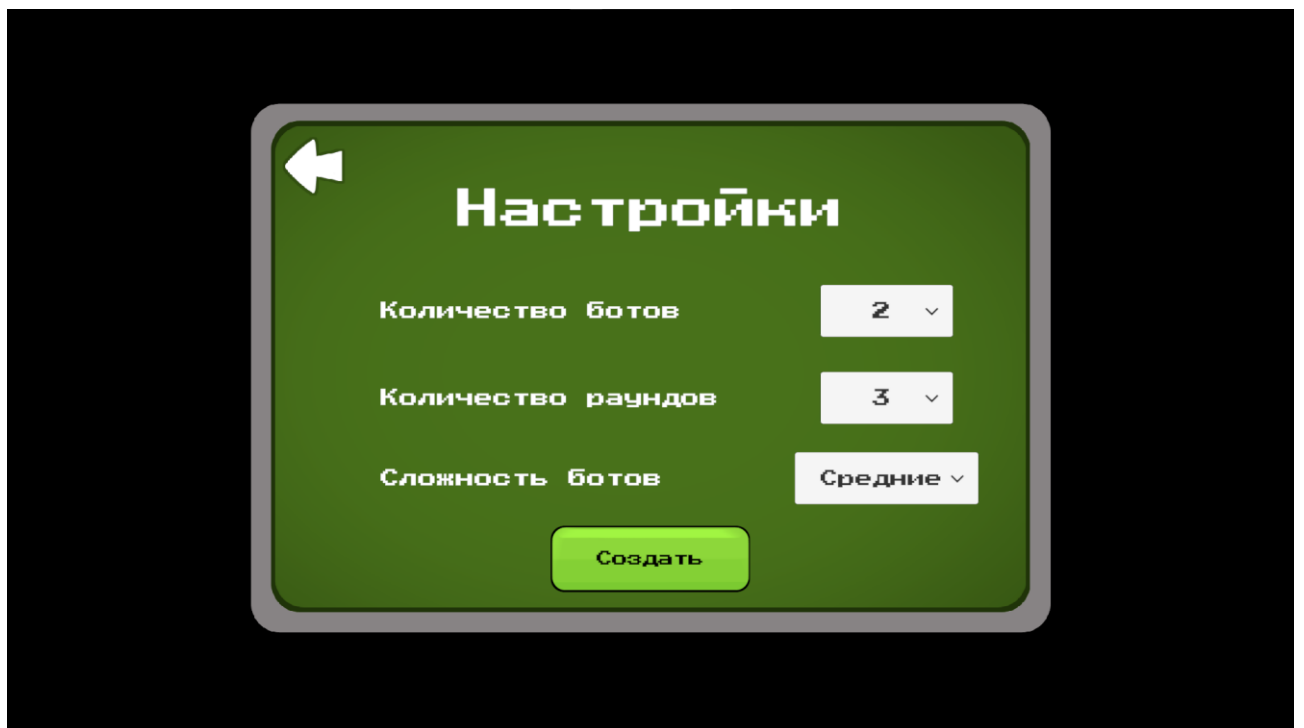


Рисунок Б.5 – Меню офлайн режима

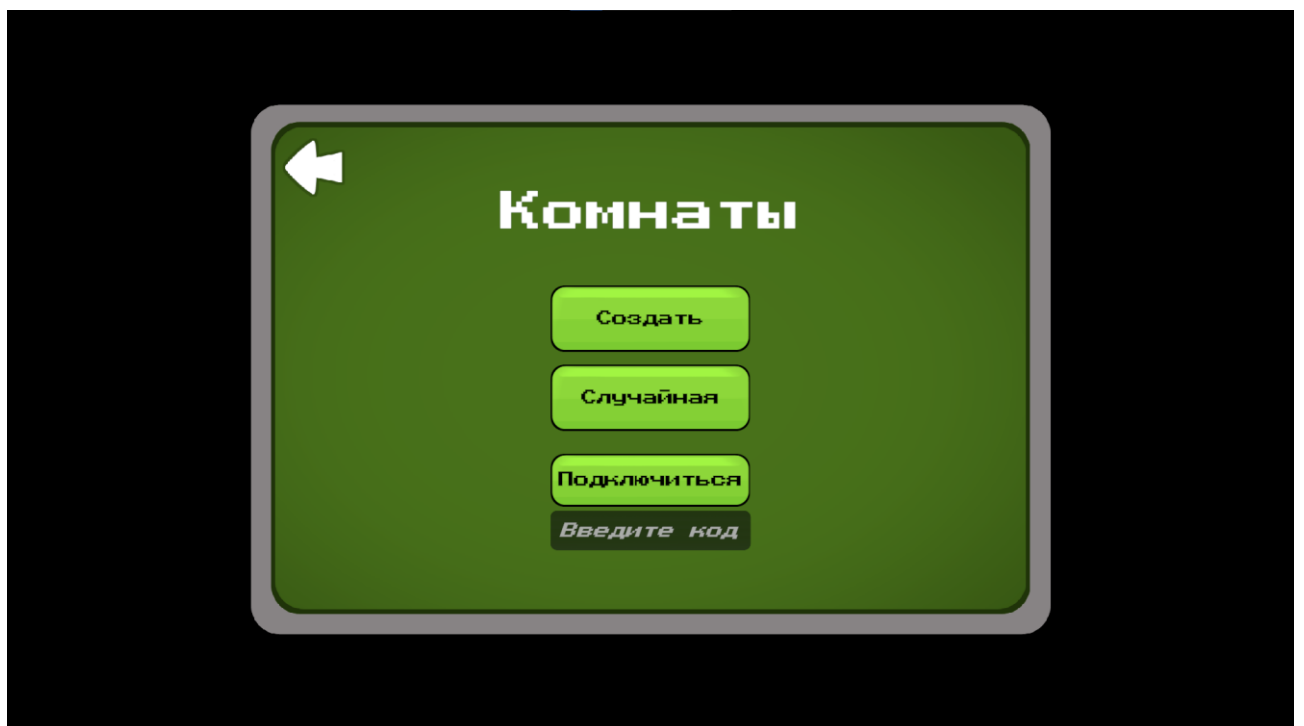


Рисунок Б.6 – Меню лобби

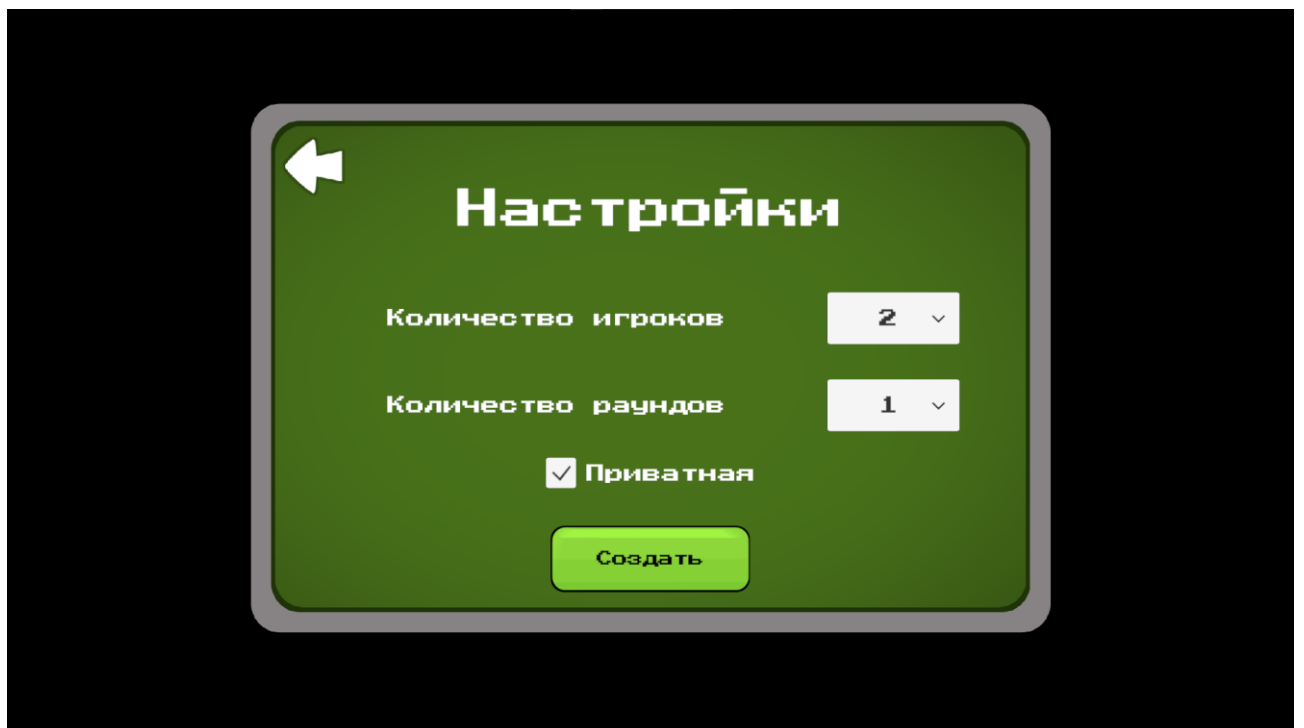


Рисунок Б.7 – Меню создания лобби

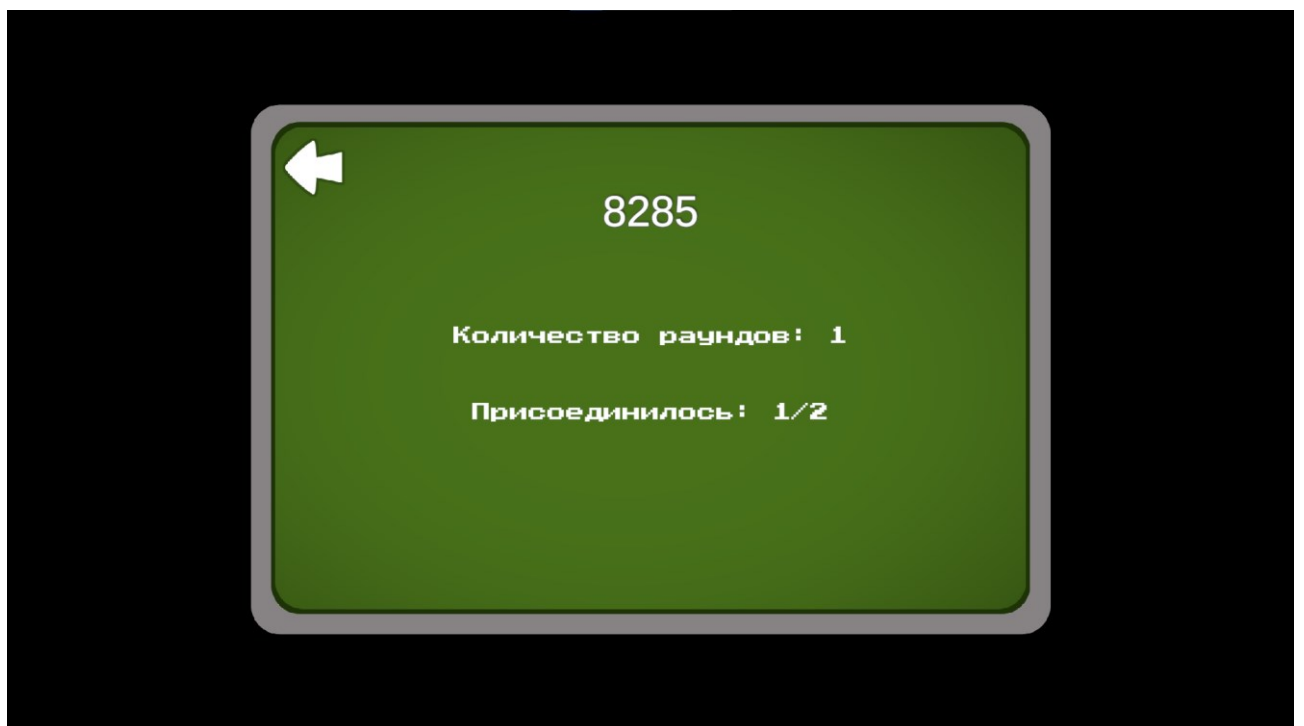


Рисунок Б.8 – Меню ожидания игроков

Приложение В
(обязательное)

```
[Test]
0 references
public void SetActiveExplosion()
{
    // Arrange
    GameObject explosionObject = new GameObject();
    Explosion explosion = explosionObject.AddComponent<Explosion>();
    AnimatedSpriteRenderer startRenderer = explosionObject.AddComponent<AnimatedSpriteRenderer>();
    AnimatedSpriteRenderer middleRenderer = explosionObject.AddComponent<AnimatedSpriteRenderer>();
    AnimatedSpriteRenderer endRenderer = explosionObject.AddComponent<AnimatedSpriteRenderer>();

    explosion.start = startRenderer;
    explosion.middle = middleRenderer;
    explosion.end = endRenderer;

    // Act
    explosion.SetActiveRenderer(middleRenderer);

    // Assert
    Assert.IsTrue(startRenderer.enabled == false);
    Assert.IsTrue(middleRenderer.enabled == true);
    Assert.IsTrue(endRenderer.enabled == false);
}
```

Рисунок В.1 – Unit-тест проверки взрыва

```
[Test]
0 references
public void StartRoundTest()
{
    // Arrange
    GameManager gameManager = new GameManager();
    int initialRound = gameManager.GetCurrentRound();

    // Act
    gameManager.StartRound();

    // Assert
    Assert.AreEqual(initialRound + 1, gameManager.GetCurrentRound());
}
```

Рисунок В.2 – Unit-тест проверки запуска нового раунда

```
[Test]
0 references
public void SavesGameSettings()
{
    // Arrange
    GameManager gameManager = new GameManager();

    // Act
    gameManager.SetPlayer(1);
    gameManager.SetRound(2);
    gameManager.SaveGameSettings();

    // Assert
    Assert.AreEqual(PlayerPrefs.GetInt("NumPlayers"), 1);
    Assert.AreEqual(PlayerPrefs.GetInt("NumRounds"), 2);
}
```

Рисунок В.3 – Unit-тест проверки сохранения локальных данных игры

Идентификатор			TU01
Заголовок			Начало нового раунда
Предусловие			Есть игроки для теста
Тестовые данные			Игра
Ожидаемый результат			Новый раунд начался и был записан новый счёт игроков
Шаг	Действие	Предполагаемый результат	Pass / Fail
1	Начало новой игры	Игра запустилась	Pass
2	Один игрок выиграл	Ожидание 2 секунды	Pass
3	Запуск нового раунда	Раунд запустился, счёт обновился	Pass
Результат			Pass

Рисунок В.4 – Тест-кейс проверки начала нового раунда

Идентификатор			TU02
Заголовок			Проверка настроек игры
Предусловие			Есть данные для настройки игры
Тестовые данные			Громкость музыки 100% и 0%
Ожидаемый результат			Громкость была изменена
Шаг	Действие	Предполагаемый результат	Pass / Fail
1	Переход в меню настройки	Меню открылось	Pass
2	Выбор громкости музыки на слайдере	Значение выбрано и громкость изменилась	Pass
3	Нажатие кнопки “выключить музыку”	Музыка выключилась	Pass
Результат			Pass

Рисунок В.5 – Тест-кейс настройки игры

Идентификатор			TU03
Заголовок			Подключение в частную комнату
Предусловие			Есть игроки для теста, игрок 1 создал частную комнату
Тестовые данные			Игра, два игрока
Ожидаемый результат			Игрок 2 зашёл в комнату
Шаг	Действие	Предполагаемый результат	Pass / Fail
1	Ввод частного ключа в поле для ввода ключа	Ключ отображается в поле для ввода	Pass
2	Нажатие на кнопку “Присоединиться”	Игрок зашёл в комнату Игрок 1 увидел, что игрок 2 зашёл	Pass
Результат			Pass

Рисунок В.6 – Тест-кейс подключения в частную комнату

Приложение Г

(обязательное)

Г.1 Листинг – Скрипт реализации взрыва

```

using Photon.Pun;
using UnityEngine;
using System.Collections;
public class Explosion : MonoBehaviourPun
{
    public AnimatedSpriteRenderer start;
    public AnimatedSpriteRenderer middle;
    public AnimatedSpriteRenderer end;

    private void Awake()
    {
        start.enabled = false;
        middle.enabled = false;
        end.enabled = false;
    }
    public void SetActiveRenderer(AnimatedSpriteRenderer renderer)
    {
        start.enabled = renderer == start;
        middle.enabled = renderer == middle;
        end.enabled = renderer == end;
    }
    public void SetDirection(Vector2 direction)
    {
        float angle = Mathf.Atan2(direction.y, direction.x);
        transform.rotation = Quaternion.AngleAxis(angle * Mathf.Rad2Deg,
Vector3.forward);
    }
    public void DestroyAfter(float seconds)
    {
        StartCoroutine(DestroyAfterDelay(seconds));
    }
    private IEnumerator DestroyAfterDelay(float seconds)
    {
        yield return new WaitForSeconds(seconds);
        try
        {
            if (PhotonNetwork.InRoom && photonView.IsMine)
                PhotonNetwork.Destroy(gameObject);
            else
                Destroy(gameObject);
        }
        catch (System.Exception ex)
        {
            Debug.LogWarning($"[Explosion] Destroy failed: {ex.Message}");
            Destroy(gameObject);
        }
    }
    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            MovementController movementController =
other.GetComponent<MovementController>();
            if (movementController != null)

```

```

        {
            movementController.DeathSequence();
        }
    }
}

```

Г.2 Листинг – Скрипт реализации лобби

```

using Photon.Pun;
using Photon.Realtime;
using UnityEngine;
using UnityEngine.UI;
using TMPPro;
using UnityEngine.SceneManagement;
public class LobbyManager : MonoBehaviourPunCallbacks
{
    public static LobbyManager Instance;
    [Header("Menu Panels")]
    [SerializeField] private GameObject menuPanel;
    [SerializeField] private GameObject lobbyPanel;
    [Header("UI Elements")]
    [SerializeField] private TMP_InputField roomCodeInputField;
    [SerializeField] private TMP_Text errorText;
    [SerializeField] private TMP_Dropdown maxPlayersDropdown;
    [SerializeField] private TMP_Dropdown roundsDropdown;
    [SerializeField] private TMP_Dropdown roundsOfflineDropdown;
    [SerializeField] private TMP_Dropdown botCountDropdown;
    [SerializeField] private TMP_Dropdown difficultyDropdown;
    [SerializeField] private Toggle isPrivateToggle;
    private const int MinPlayersToStart = 2;

    private void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
        }
        else Destroy(gameObject);
    }
    private void Start()
    {
        if (PhotonNetwork.IsConnected) PhotonNetwork.Disconnect();

        PhotonNetwork.ConnectUsingSettings();
    }

    public override void OnConnectedToMaster()
    {
        PhotonNetwork.JoinLobby();
    }
    public void OnStartOfflineGame()
    {
        int rounds = roundsOfflineDropdown.value + 1;
        int botCount = botCountDropdown.value + 1;
        PlayerPrefs.SetInt("OfflineRounds", rounds);
        PlayerPrefs.SetInt("OfflineBotCount", botCount);
        PlayerPrefs.SetInt("OfflineBotDifficulty", difficultyDropdown.value);
        PlayerPrefs.Save();

        SceneManager.LoadScene("OfflineGame");
    }
}

```



```

    }

    public void CreateRoom()
    {
        string roomCode = Random.Range(1000, 9999).ToString();

        if
        (!byte.TryParse(maxPlayersDropdown.options[maxPlayersDropdown.value].text, out
        byte maxPlayers))
        {
            Debug.LogError("Invalid max players value.");
            return;
        }

        int rounds = roundsDropdown.value + 1;

        RoomOptions options = new RoomOptions
        {
            MaxPlayers = maxPlayers,
            IsVisible = !isPrivateToggle.isOn,
            CustomRoomProperties = new ExitGames.Client.Photon.Hashtable
            {
                { "roomCode", roomCode },
                { "rounds", rounds }
            },
            CustomRoomPropertiesForLobby = new[] { "roomCode", "rounds" }
        };

        PhotonNetwork.CreateRoom(roomCode, options);
    }

    public override void OnCreatedRoom()
    {
        ShowLobbyPanel();
    }

    public void JoinRoomByCode()
    {
        string code = roomCodeInputField.text.Trim();
        if (string.IsNullOrEmpty(code))
        {
            errorText.text = "Впишите код комнаты";
            return;
        }
        PhotonNetwork.JoinRoom(code);
    }

    public override void OnJoinedRoom()
    {
        PhotonNetwork AutomaticallySyncScene = true;
        ShowLobbyPanel();
    }

    public void JoinRandomRoom()
    {
        PhotonNetwork.JoinRandomRoom();
    }

    public override void OnJoinRandomFailed(short returnCode, string message)
    {
        errorText.text = "Нет свободных комнат";
        Debug.LogWarning("No random rooms available.");
    }

    public override void OnMasterClientSwitched(Player newMasterClient)

```

```

{
    PhotonNetwork.LeaveRoom();
    UnityEngine.SceneManagement.SceneManager.LoadScene("Menu");
}
public override void OnJoinRoomFailed(short returnCode, string message)
{
    errorText.text = "Ошибка подключения к комнате";
    Debug.LogError($"Failed to join room: {message}");
}

public override void OnLeftRoom()
{
    ShowMenuPanel();
}
public override void OnDisconnected(DisconnectCause cause)
{
    ShowMenuPanel();
}
private void ShowLobbyPanel()
{
    menuPanel.SetActive(false);
    lobbyPanel.SetActive(true);
}

private void ShowMenuPanel()
{
    lobbyPanel.SetActive(false);
    menuPanel.SetActive(true);
}
}

```

Г.3 Листинг – Скрипт офлайн игры

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
using TMPro;
public class OfflineGameManager : MonoBehaviour
{
    [Header("Prefabs & Spawn Points")]
    public GameObject playerPrefab;
    public GameObject botPrefab;
    public Transform[] spawnPoints;
    [Header("UI")]
    public TMP_Text playerScoreLabel;
    public TMP_Text botScoreLabel;
    public GameObject endGamePanel;
    public TMP_Text endGameText;
    public float endDelay = 1f;
    private static bool s_initialized = false;
    private static int s_totalRounds = 1;
    private static int s_roundsPlayed = 0;
    private static int s_playerScore = 0;
    private static int s_botScore = 0;
    private GameObject player;
    private List<GameObject> bots = new List<GameObject>();
    private bool gameEnding = false;
    void Start()
    {
        if (!s_initialized)

```

```

    {
        s_totalRounds = PlayerPrefs.GetInt("OfflineRounds", 1);
        s_roundsPlayed = 0;
        s_playerScore = 0;
        s_botScore = 0;
        s_initialized = true;
    }

    UpdateScoreLabels();
    SpawnAll();
    endGamePanel.SetActive(false);
    gameEnding = false;
}

void Update()
{
    if (gameEnding) return;

    bool playerAlive = player != null && player.activeSelf;
    bool anyBotAlive = bots.Exists(b => b != null && b.activeSelf);

    if (!playerAlive || !anyBotAlive)
    {
        gameEnding = true;

        bool playerWon = playerAlive;
        if (playerWon) s_playerScore++;
        else s_botScore++;

        s_roundsPlayed++;
        UpdateScoreLabels();

        StartCoroutine(NextRoundRoutine(playerWon));
    }
}

private IEnumerator NextRoundRoutine(bool playerWon)
{
    yield return new WaitForSeconds(endDelay);

    if (s_roundsPlayed < s_totalRounds)
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }
    else
    {
        endGamePanel.SetActive(true);
        endGameText.text = playerWon
            ? "Вы выиграли матч!"
            : "Вы проиграли матч!";
        s_initialized = false;
    }
}

private void SpawnAll()
{
    bots.Clear();

    player = Instantiate(playerPrefab, spawnPoints[0].position,
Quaternion.identity);
    player.tag = "Player";
    var pm = player.GetComponent<MovementController>();
    var bc = player.GetComponent<BombController>();
    if (pm != null) pm.isOfflineMode = true;
    if (bc != null) bc.isOfflineMode = true;

    int botCount = PlayerPrefs.GetInt("OfflineBotCount", 1);

```

```

int diffIdx = PlayerPrefs.GetInt("OfflineBotDifficulty", 0);

for (int i = 0; i < botCount; i++)
{
    int idx = Mathf.Clamp(i + 1, 1, spawnPoints.Length - 1);
    var bot = Instantiate(botPrefab, spawnPoints[idx].position,
Quaternion.identity);
    bot.name = $"Bot_{i + 1}";
    var botCtrl = bot.GetComponent<BotController>();
    if (botCtrl != null) botCtrl.difficultyIndex = diffIdx;
    pm = bot.GetComponent<MovementController>();
    bc = bot.GetComponent<BombController>();
    if (pm != null) pm.isOfflineMode = true;
    if (bc != null) bc.isOfflineMode = true;

    bots.Add(bot);
}
}
private void UpdateScoreLabels()
{
    playerScoreLabel.text = s_playerScore.ToString();
    botScoreLabel.text = s_botScore.ToString();
}
public void LoadMenu()
{
    s_initialized = false;
    SceneManager.LoadScene("Menu");
}
}

```